

A Systematic Literature Review on Test Case Prioritization Techniques

Harendra Singh, Department of Computer Science and Engineering, Dr. A. P. J. Abdul Kalam Technical University, Lucknow, India

Laxman Singh, Department of Electronics and Communication Engineering, Noida Institute of Engineering and Technology, Greater Noida, India*

Shailesh Tiwari, Krishna Engineering College, Ghaziabad, India

ABSTRACT

Regression testing (RT) provides confidence for any software to work properly after incorporating modifications in the software functionalities. The amendments in the software are visible due to the evolution or the adoption of the new functionalities in the software. New test cases might be added or removed from the test suit during the RT. TCP seeks to provide an execution order of test cases when test cases are large in number for the detection of faults at an early stage. This paper provides classification of available TCP techniques that are examined based on the six research questions having higher relevance regarding the TCP. To carry out a systematic literature review (SLR) on TCP techniques, the authors extracted a large number of papers from appropriate repositories of journals, conferences, workshops, and symposiums using various search keywords. Each TCP approach has its own limitations, potential value, and advantages, which are covered in this SLR in considerable detail.

KEYWORDS

Regression Testing, Software Engineering, Software Testing, Test Case Prioritization

1. INTRODUCTION

Regression activity ensures the proper functioning of the working software under trial during the development phase (Li et al., 2007). Regression testing is usually employed when any alteration takes place in the software. These alterations are frequent due to customer's requirement, availability of the new and better algorithms for computing task as well as owing to the revamping in design of the software for reducing the complexity of existing software. Regression testing is a critical and expensive task (Li et al., 2007). Regression testing tends to be an integral part of the software system, and hence plays a pivotal role in enhancing its functionality (Do, 2016; Elbaum et al., 2014; Hao, Zhang, & Mei, 2016; Liang et al., 2018; Ralph, 2016). Perf Ranker is claimed to be the first approach in performance regression testing for collection intensive software (Elbaum et al., 2014). Software testing is as an expensive phase owing to being a repetitive task (Hao, Zhang, & Mei, 2016), therefore, consistent efforts need to be made by software developers to eliminate the testing phase owing to financial and time constraints.

DOI: 10.4018/IJSI.312263

*Corresponding Author

Test case prioritization techniques are applied in continuous integration (CI) environments (Liang et al., 2018). The CI server monitors make alteration in the code repository (CR) and ensure whether changes have occurred and executed the software build and the test cases with respect to the change taken place in CR. Software tends to evolve with time and hence the 33% of total cumulative cost needed for software development is invested over regression testing (Chittimalli & Harrold, 2009; Mostafa et al., 2017). Test case selection, minimization, and prioritization are accountable for regression testing of a software (Jeffrey & Gupta, 2007; Yoo & Harman, 2007).

Test suit minimization is used to eliminate the test cases from the test suit to minimize the number of those test cases, which are required to run the software (Jeffrey & Gupta, 2007). The minimization is sometimes called test suit reduction. The elimination of test cases is said to be permanent in test case reduction process.

Test case selection approach is used to execute those test cases, which are aware of modification and needs to be executed. Test case selection approach mitigates the number of test cases by selecting the test cases that have relation with the modified test cases (Li et al., 2007).

The evidence-based software engineering (EBSE) practices finds applicability for research in test case prioritization (TCP) (Kitchenham & Brereton, 2009). TCP orders the test cases in such a way that it achieves optimization based on some properties (i.e., time and cost) of preference (Kitchenham, 2004). Prioritization techniques (Li et al., 2007) create a sequence of execution of test cases based on matrix using fitness function. For instance, If T is a test suit and PT is the permutation of the T . Let f be a function from PT to a real number (Li et al., 2007). The problem is to find $T \in PT$ such that $(\forall T' (T' \in PT) (T' \neq T) [f(T) \geq (T')])$.

Here, function f is used to find T from all possible permutation (i.e., from PT) based on the value that makes it better member than others.

Table 1 shows the approaches and their elements for regression testing. Selection, Minimization and Prioritization are three approaches, and each has its own strategy, capacity, and limitations. The minimization aims at reducing the test suit by eliminating the obsolete test cases from the original test suit. However, new test cases are added for new functionalities with the amalgamation of the latest build. Test case selection technique selects the test cases that cause a modified program to yield different output other than the original program. The test case prioritization technique generates the best sequence for test case execution, by which the modified code can be tested optimally against the faults in the software. In test case prioritization, every new test case ought to be added in the test suit for execution. To observe abnormality in the software, which occurs generally due to the modification in the software, new test cases are crucial.

Over the past 20 years, numerous approaches have been introduced by the researchers for Test Case Prioritization. Various reviews in respect to TCP are available in literature based on different criteria. In general, there are three most popular criterions viz., context, effect, and information, which

Table 1. Approaches for Regression testing

Element	Approaches of Regression Test		
	Selection	Minimization	Prioritization
Strategy	Test case involvement in modification	Eliminate the test cases	Permutation of test cases using prioritization and ordering
Capacity	Valuable in selecting the test case of modified	Valuable in downsizing the test suit.	If new test cases are considered in the test case permutation.
Limitation	New test cases may not be present for modified code.	Test case may not be aware of the modification.	Takes a huge time if the size of the test suit is large.

are used for regression testing (Ali et al., 2019). B. Kitchenham et al. (Kitchenham & Brereton, 2009) presented a systematic literature review on the latest approaches and techniques used for TCP. The SLR is the special study of evidence in research that not only provides a summary of current proof of research questions, but also strengthen the research.

In this SLR, section 2 presents the previous studies related to TCP approaches. Section 3 presents the studies selected to direct this SLR. Section 4 includes results and discussion based on research questions. Research Findings are then discussed in section 5. The threats of validity with regards to SLR has been elaborated in section 6. In section 7, we eventually conclude the work.

2. REGRESSION TESTING BACKGROUND

This section presents previous studies covering TCP approaches for regression testing. First of all, we provide a brief overview of the literature review (LR) papers, mapping studies (MS) and survey papers, which have already been covered in the literature related to TCP. Details of total nine (i.e., 02-SLR, 02-Mapping studies and 05-survey) papers related to test case prioritization and regression testing are presented in Table 2.

In (Singh, 2012), Y. Singh presented 91 studies in total on SLR, out of which, 49 were based on the initiation of different approaches, 14 studies were related to analysis of previous studies and two were based on the augmentation of old studies. Eight broad approaches were covered and analyzed in this particular SLR. These approaches were based on genetic algorithm, coverage, requirement, history, modification, composite, fault and many more as an integral part of other categories. In M. Khatibsyarbini et al. (2018) studied 100 papers in total covering 74 papers on different approaches and 69 papers about primary studies. Rests of the papers were based on software engineering and regression testing. In (Khatibsyarbini et al., 2018), authors also discussed some additional approaches like search-based algorithm, Bayesian-Network, cost effective, Multi-Criteria, and Topic model-based approaches. In search-based approaches, GA, Greedy, ACO and String based approaches were primarily investigated. In (Kumar & Singh, 2014), Kumar and Singh categorized the TCP process based on the cost and coverage techniques. They covered 19 papers in total focusing mainly on coverage based and cost-effective based techniques as stated earlier. In this paper, authors suggested that if new technique is applied at an early stage in the software development life cycle, it reduces the development cost of the software.

Table 2. Regression Test Prioritization (RTP) studies

Research Type	Focus of Study	Research References	Publication year	Year Covered
SLR	Test Case Prioritization	Singh et al.	2012	1997-2011
	Test Case Prioritization	Md. Khatibsyarbini et al.	2018	2002-2016
Mapping	Test Case Prioritization	Catal and Mishra	2013	2001-2011
	Test case prioritization in continuous integration	Jackson A. et al.	2020	1999-2020
Survey	TCP Literature Survey	Yoo and Harman	2012	1977-2009
	TCP Literature Survey	Kumar and Singh	2014	NA
	TCP Literature Survey	Kiran and Chandraprakash	2015	NA
	Survey on TCP	Yiling lou et al.	2018	1997-2016
	Survey on Achievements and challenges of TCP	Dan HAO et al.	2016	1997-2016

As stated earlier, in this paper, we covered two systematic mapping studies, which are presented by Catal et al. (2012) and Parado et al. (2020) in year 2012 and 2020, respectively. The mapping study given in (Catal & Mishra, 2012) mainly focus on TCP research, and the TCP techniques. In literature (Catal & Mishra, 2012), authors searched and investigated the reviews that were published between year 2001 to 2011. Mapping study reported in (Jackson, 2020) provides information about the main characteristics and the evaluation aspects of TCP techniques. In their study total 35 research studies were accessed, which were based on the goals, kind of TCP Techniques used the problem under study and the evaluation measure. This study involves 80% of the approaches based on history (i.e., failure and test execution history). A minor survey conducted by Kiran et al. (2015), suggested that cost and other parameters might be considered to enhance the capability of TCP approaches further. This idea was suggested based on the 90 research papers accessed from the IEEE publications. In (Lou et al., 2019), authors investigated 191 articles on test case prioritization techniques from year 1997 to 2016 covering six different aspects such as algorithms, criteria, measurement, constraints, empirical studies, and scenario. To anticipate the best prioritization approaches, more reviews were needed to be taken into consideration owing to the huge increase in number of new research articles produced as of then.

D. Hao et al. (2016) presented a review paper on the history of test case prioritization and its representation. They briefly reviewed numerous state of art methods relied upon test-case prioritization techniques with five different aspects viz., coverage, criteria, prioritization, measurement, constraints, and application scenarios. In their analysis report, they rendered challenges in the TCP including the measurement, evaluation, and practical applicability. Yoo and Harman (2007) discussed about an important survey article in the field of regression testing. This paper focuses on the area of test suit minimization (TSM), test case selection (TCS) and TCP. Based on the analysis of total 159 papers, the survey in regard to TCP was classified in different categories based on the following criterion such as coverage code, requirement model, probabilistic calculation, cost awareness and many minor approaches.

Table 3 presents the research finding summary of different research papers including survey papers, SLR, and mapping studies.

3. RESEARCH METHODS

Figure 1 presents the different steps of review process adopted in literature for TCP approaches (Kitchenham & Brereton, 2009), (Kitchenham, 2004), (Achimugu et al., 2014).

The review process shown in Figure 1 includes various steps such as research questions, repository selection, searching blueprint (i.e., search string and searching process), selecting studies (i.e., criteria for inclusion and exclusion), data extraction and synthesis. In this SLR, we have taken total six questions to meet the objective of the present study. After execution of the first step, valid repository was searched and investigated to find the relevant papers therein. We then applied search strategy to begin the selection process of the relevant studies through search string and searching process based on the research questions we used. Quality criterion was also applied to select the relevant studies. In the last phase, data extraction and synthesis steps were used to extract the primary studies for this SLR as illustrated in Figure 2.

3.1 Research Questions With Reasons

In current SLR, a detailed investigation is done about recent experimental evidence, which we came across during the review process. The prime objective of this review is to improve the ability of the current TCP approaches. To complete the review task efficiently, we prepared six questions with reasons as listed in Table 4. The prime goal of the present study is to condense the current state of art TCP approaches by suggesting the answers of those questions, which are listed in Table 4.

Table 3. Research finding summary

Referenced study	Findings Covered	Common findings	Additional Findings
Singh et al.	Several TCP approaches and empirical evidence Gaps in using tools, artifacts, and metrics	Empirical evidence Gaps in metrics	Empirical evidence for new approaches Gaps in use of TCP artifact use.
M. Khatibsyarbini et al.	Various TCP approaches Usages of Techniques in different scenarios Gaps in using tools. Gaps in artifact and metrics.	Empirical evidence Cavity in used metrics.	Empirical proof of new approaches. Difference between TCP artifact used. Grounds of creation of uncovered metrics.
Catal and Mishra	TCP approaches trends. TCP publication trends. TCP dataset trends.	TCP approach trends Metric trends in papers	Why we tend to follow the TCP approaches? Why are evaluation metrics used?
Jackson A et al.	TCPCI Approaches evaluated. Report's evaluation aspects.	TCP approaches trends. Report's evaluation aspect	Why is a particular approach used? Ground's metrics used
Yoo and Harman	TSM, TCS and TCP overview 4 approaches of TCP in detail Metric types for TCP	Detail outline of TCP approaches	Comprehensive outline of other approaches of TCP Reason given for creation of uncovered metrics.
Kumar and Singh	Coverage and cost oriented techniques outline.	Outline of approaches (in terms of techniques, strategies and algorithms)	Comprehensive outline of other recent TCP approaches Logic behind the use of evaluation metrics
Kiran and Chandraprakash	Overall overview for TCP Strategies, algorithms, and metrics for various TCP techniques.	Explained TCP approaches in terms of techniques, algorithms, and strategies.	Overview of recent TCP approaches.
Yiling lou et al. [20]	Extensive survey of TCP. Survey based on six aspects. Limitations of TCP approaches	Comprehensive study of TCP approaches. Issues related to approaches	Includes empirical evidence of new approaches. More categories covered.
Dan HAO et al.	TCP survey based on achievements and challenges. Five categories of given TCP approaches given	Coverage of TCP approaches based on issues are covered. More TCP approaches covered	Each approach covered with issues. Comprehensive study of logic behind the approach.

The above-mentioned research questions are relatively connected and can provide the answers of supplementary findings of the SLR. Table 5 provides the supplementary findings of the research questions with their associated significance.

The research questions enclose the supplementary findings; thereby help to achieve the goal of SLR undertaken for study.

Figure 1. Different steps of Review Process

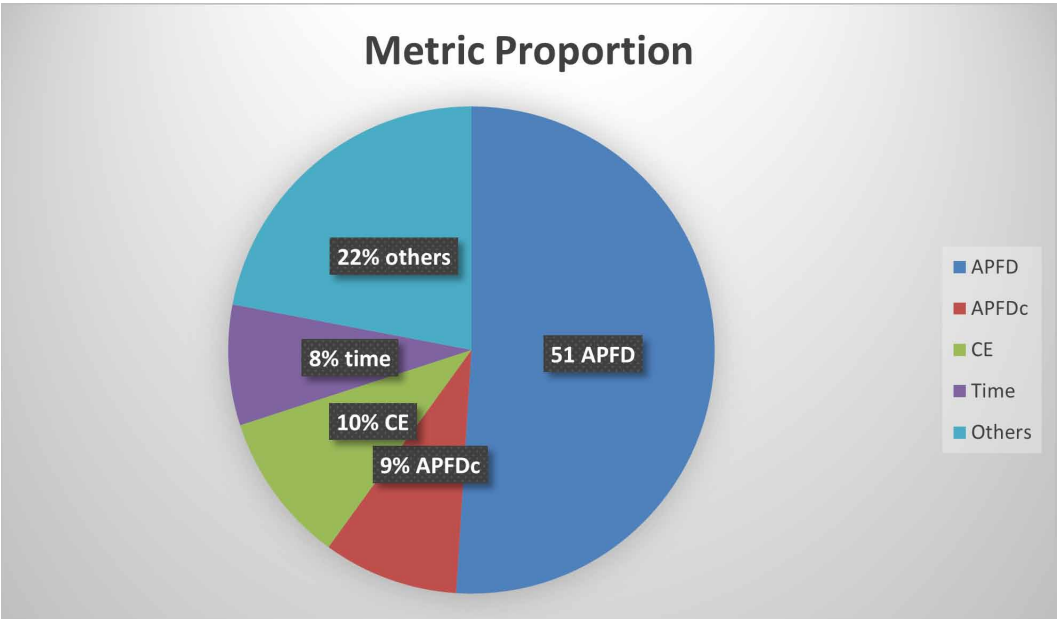


Figure 2. The Action Plan

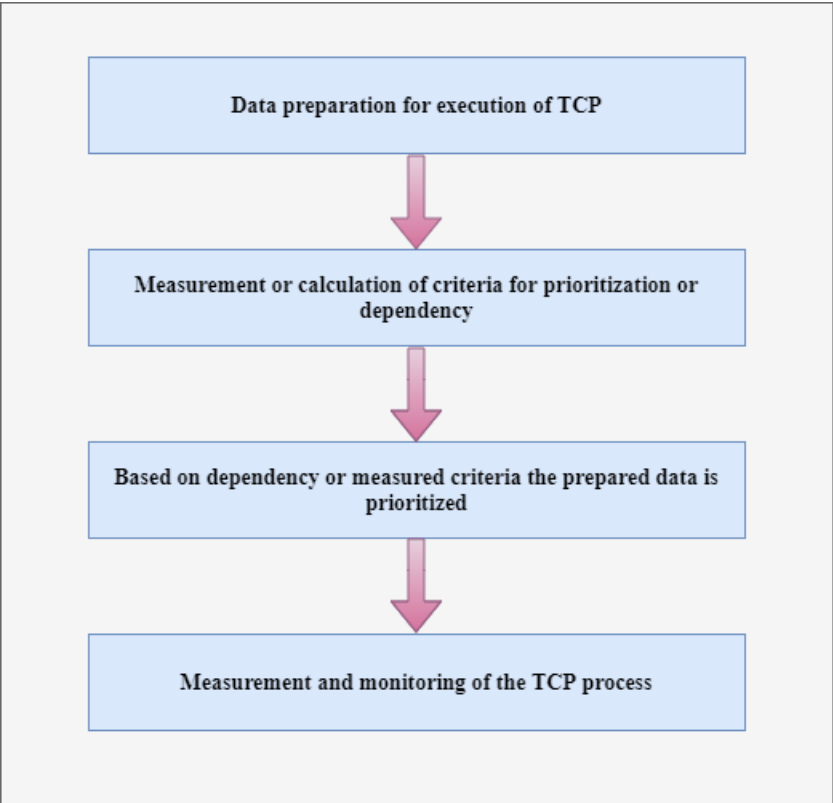


Table 4. Research questions with reasons

Research Questions	Research Question Statement	Reason
Q1:	What are current research trends and empirical evidence in TCP in Regression Testing?	Characterization of current domain of TCP to obtain available approaches.
Q2:	How are the TCP approaches distributed?	In past years, what are different kinds of algorithms used in different approaches?
Q3:	What are positives and negatives of current TCP approaches?	The knowledge about different functioning of TCP approaches.
Q4:	What is the way of applying the TCP approach and how does this affect the result?	The basis of improvement is strength and weakness.
Q5:	How is TCP used in Regression Testing?	This helps to illustrate execution process of TCP for various approaches.
Q6:	How are metrics and artifacts involved in TCP?	This question helps other researchers to choose suitable metric.

Table 5. Research questions and supplementary findings

Research Questions	Supplementary findings with their associated significance
Q1, Q2	TCP approaches distribution with logic; it provides insight in recent trends of TCP in Regression Testing.
Q3, Q4	The capacity and limitation of the existing approaches; it provides the different functionalities to the TCP approaches and informs about the further improvement in TCP approaches.
Q5	The TCP process illustrates the basic execution of TCP process.
Q6	The metric used for TCP; this informs us about available metrics and artifacts regarding a particular TCP approach.

3.2 Strategies Used

SLR comprises of three stages, which are given as follows:

1. Selecting the Literature repository.
2. Identification of strings for searching out the research papers.
3. The selection procedure to select the specific study.

3.2.1 Selection of the Literature Repository

The authors begin the process of finding the studies by inserting “Test Case Prioritization” as the key string on Google scholar database. Google scholar database returned thousands of research papers. From the search, author selected the papers related to TCP and primarily focused on the collection of primary studies. For this SLR, the collected papers have been obtained from the following sources:

1. ACM Digital Library
2. Wiley Online Library
3. Google scholar
4. IEEE Explore
5. Elsevier
6. Springer

The search provided a good number of conferences, symposium, and journal articles. In addition, research articles, various literature review as well as short communication papers related to TCP regression testing were also explored.

3.3 Identification of Search Strings

The information gathered from the relevant sources through the extensive search was turned into SLR, for which we used the following steps:

- 1. Determine the significant terms based on RQ.
- 2. Replace the significant terms by their equivalent terms.
- 3. Selection of keywords for accessing the relevant studies.
- 4. Usage of Boolean operators as link between terms.

3.4 Selection Process for Research Studies

To provide high impact review, the SLR needs to be conducted in an efficient and time bound manner. The test case prioritization is used as the exact phrase for finding research material. At the time of selection of repositories, the process of SLR was initiated. First of all, several popular repositories of TCP were identified. Afterwards, exhaustive search process was performed on all the selected repositories, followed by selection of papers for primary studies. Fig. 3. illustrates the process of searching and selecting the papers for the SLR, that we conducted.

In the search stage, out of the total 500 studies, only the most significant studies were selected, which justified our research purpose as can be seen in Fig. 3. Initially, we adopted inclusion and exclusion criteria for all the studies acquired through the search process. Table 6 presents the inclusion and exclusion criterion for all the papers that were collected. In our study, we included the papers that were printed and issued in English and rest of the papers, we excluded from the study. The abstract of every paper was studied carefully, based upon which, only the papers found suitable

Figure 3. Search and selection of research papers

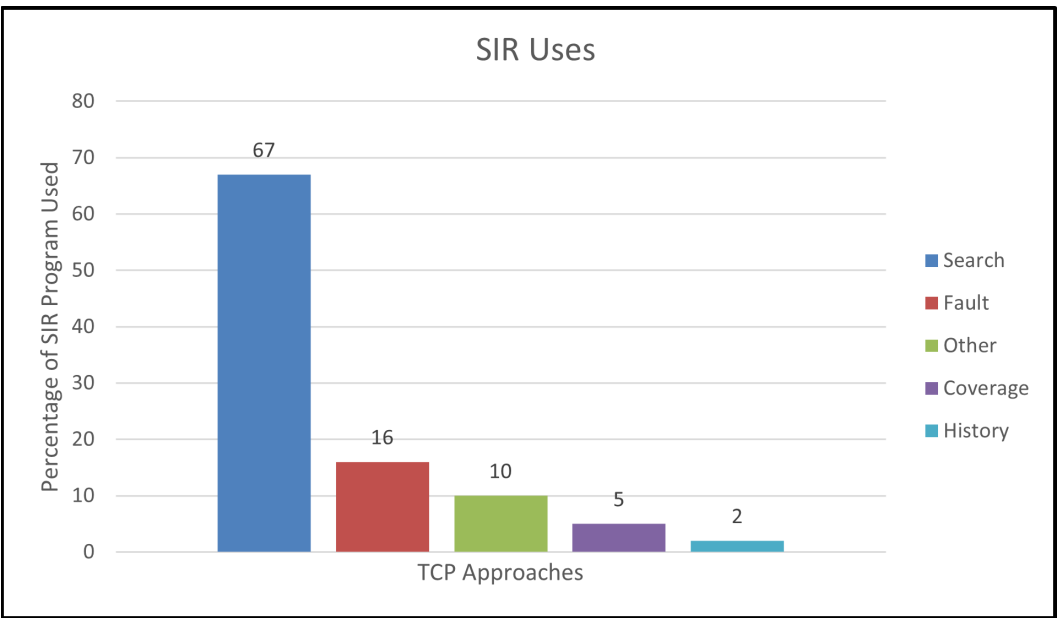


Table 6. Exclusion and Inclusion Criteria

Exclusion Criteria	Inclusion Criteria
Publication language other than English.	Publication language is English.
TCP approaches not given in the papers.	Paper focuses on TCP approach.
No Bibliographic information available in research studies.	Paper with bibliographic information (1999-2021)
In case of duplicate papers, only recent papers were considered and non-peer reviewed were not considered.	Paper answers any of the research questions.

were included in this SLR. In case of duplicate papers, only the recent ones were given preference for inclusion in this study.

After adoption of inclusion and exclusion criterion, we followed the quality assessment step, in which, we used the weighting approach to examine the appropriateness of the study. As stated earlier, six questions were articulated to evaluate the applicability, reliability, and the compressibility of the collected research studies. Using the weighting approach, we assigned the weights to each study based on the evaluation of the above parameters (i.e. applicability, reliability, and the compressibility) as No=0, partly=0.5 and yes=1. Using this quality assessment, only 91 papers were chosen for study and remaining papers got rejected. Scores of the papers are available in Table 12, which is provided in appendix section.

After accomplishment of all the steps, we found only 91 studies to have the capacity of answering all the six research questions considered in this paper.

3.5 Method of Data Extraction and Synthesis

Data synthesis is the way to simplify the presentation of the evidence from nominated papers that makes data extraction process easier. It is done with the aim of getting response of all the research questions. Table 8 provides the content measure of the selected papers.

Primary studies have been synchronized with measurement process to substantially clarify the object of the paper. Synchronization of primary studies with the measurement process was found useful to extract the crucial information required for answering the research questions. The mapping of the synthesized data and the research question is illustrated in Table 9.

4. RESULT AND DISCUSSION

In this section, Firstly, results are presented according to the research questions designed for the study. Subsequently, we presented the summary of primary studies, followed by answers received in response to the research questions.

Table 7. Quality Questions

	Quality assessment criteria
1	Is the research objective precise?
2	Was the design of experimental strategy appropriate?
3	Was planned approach clear?
4	Was conducted experiment a controlled experiment or done on a case study?
5	Whether scholar community was enhanced by exploration?

Table 8. Accessing the contents

Selected papers	Explanation
Type of Paper	Symposium, workshop, conference, and journal article
References in paper	Publication year, title of paper and sources
Focus of paper	Complications, inspiration, purpose, and idea of paper
Domain	Situation depiction
Constraint	Future improvement and limitation of the study
Methodologies	Literature survey, reviews, experimentation, and case study

Table 9. Research questions and data collection

Research Questions	Description related to Data Extraction
RQ1	Kind of bibliography, studies, and regression testing.
RQ2	The type of solution of prioritization approach to conduct technique.
RQ3	Purpose of each approach, strategy, and formal hypothesis.
RQ4	Threat to validity and advantages.
RQ5	Data collection methods, process, document type and effectiveness of TCP techniques.
RQ6	Type of software artifacts and metrics used.

4.1 Summary of Primary Studies

91 studies undertaken to conduct this SLR are comprised of the following: (i) 59 articles from journals, (ii) 23 papers from conference, (iii) 2 papers from workshop, and (iv) 7 papers from symposium. Figure 4. presents an accumulated percentage of the papers studied to conduct this SLR.

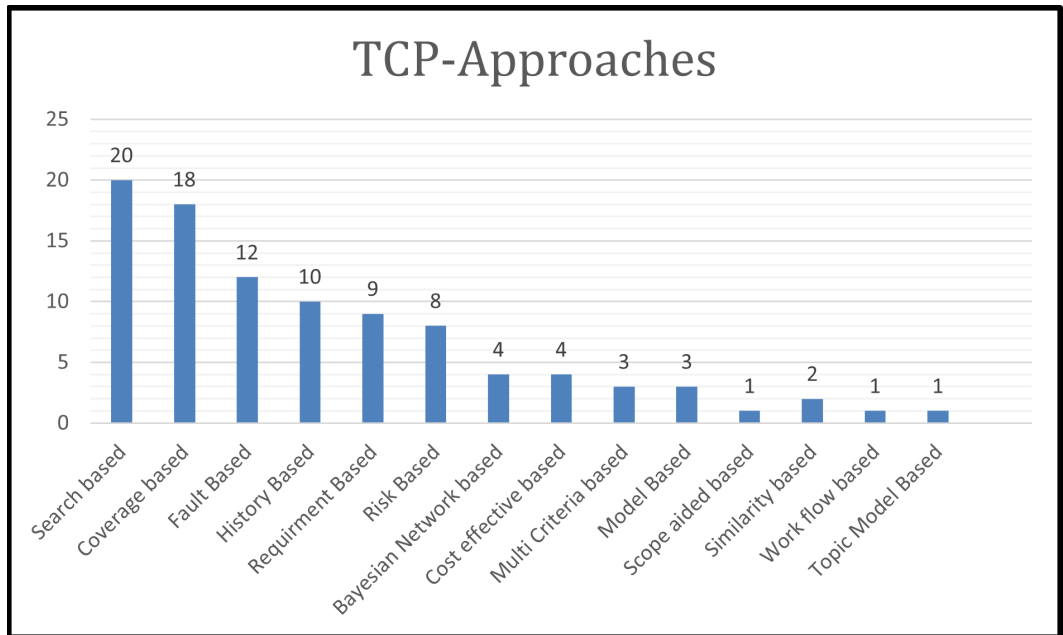
4.2 Research Trends of TCP Techniques in Regression Techniques - RQ 1

In the literature, the test selection has been done based on the test case prioritization. After year 1999, TCP techniques started gaining popularity in broader context. RQ1 determines an aspect of the current research trends in TCP techniques. Fig. 5. Illustrates the rise in the number of papers published in literature between 2003 to 2020 indicating the importance of TCP research concept in regression testing.

It is difficult to determine the importance of test case for fault detection in real life. Hence, various TCP approaches are available in recent studies, which can be employed effectively to enhance the fault detection capability of the system in many ways. These include eight broad categories as illustrated in (Kumar & Singh, 2014). The approaches have been grouped based on the available resources (i.e., Test cases, and requirement), expected output (i.e., average percentage of faults detected APFD, and time execution) and the selected phase. For instance, requirement-based TCP approach can be initiated during requirement phase. Fig. 6. depicts the classification of available TCP approaches.

The Fig. 6 also demonstrates the grouping of different dimensions of the TCP approaches. These approaches have been reported in literature in recent years and have some common characteristics (Catal & Mishra, 2012). Each approach adopts the different process for regression testing.

Figure 4. Percentage of accumulated studies used to conduct the SLR



4.3 Distribution of Approaches in TCP Techniques - RQ 2

The second research question refers to those categories, which have been profoundly suggested by researchers. Fig. 7 presents the distribution of various approaches available for TCP processes. The less popular TCP approaches are considered under the umbrella of other categories. For instance, topic-model based (Thomas et al., 2012), (Aman, 2018), cost-effective based, multi-criteria based (Khanna et al., 2019; Khanna et al., 2018; Sampath et al., 2013; Sanchez et al., 2014; Strandberg & Sundmark, 2016), workflow based (Mei, Chan, Tse et al, 2015) and other miscellaneous approaches (Fang et al., 2014; Flemstrom et al., 2018; Huang et al., 2017; Korel et al., 2007; Miranda & Bertolino, 2017; Mohd-Shafie et al., 2020; Tahat & Korel, 2017) are treated as sub-categories as shown in Figure 6. For an easier analysis, Figure 6 also represents the hierarchy of the TCP approaches.

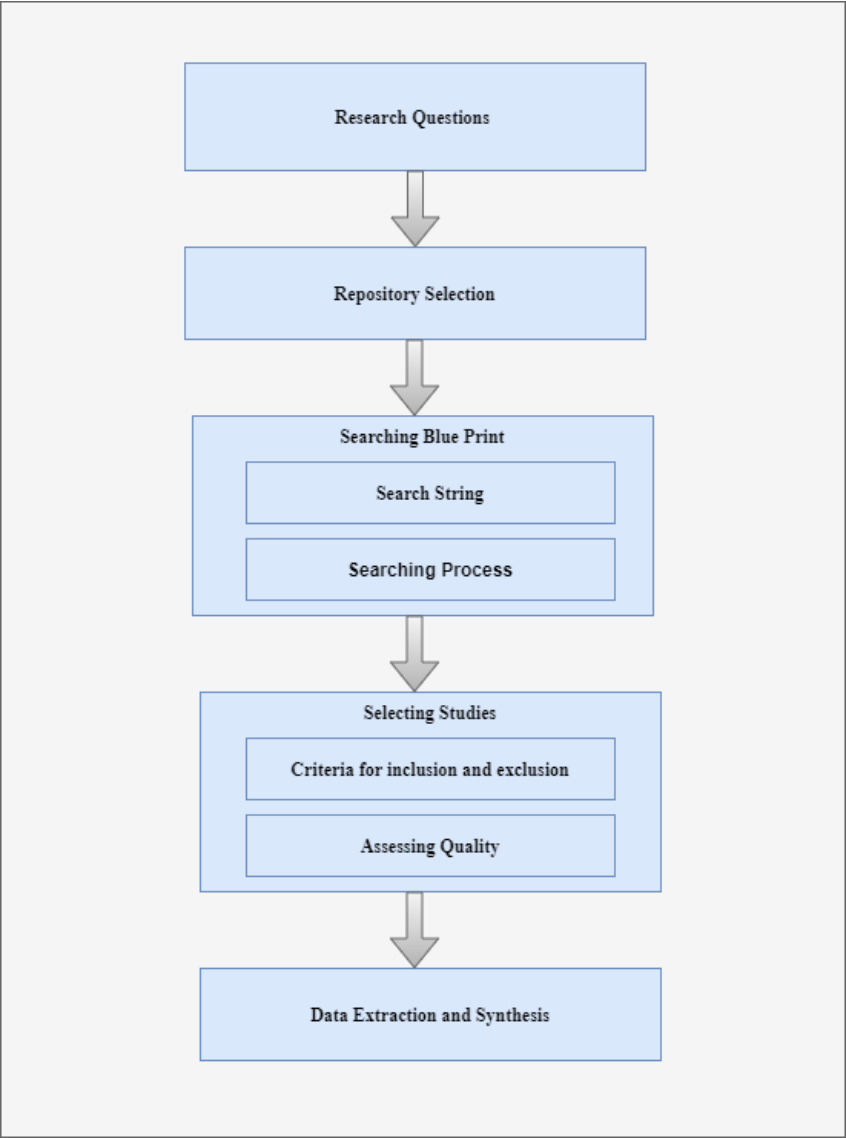
We found huge number (i.e., 21%) of the search-based approaches during collection of studies. Some of them are as follows: Genetic algorithms (), Greedy algorithms (Chi et al., 2020), (Li et al., 2007), multi objective based algorithm (Wang & Ali, 2016), ant colony optimization-based algorithms (Solanki et al., 2016)-(Bian et al., 2017), string distance-based algorithm (Ledru et al., 2012) and other miscellaneous algorithms (Eghbali & Tahvildari, 2016).

Coverage based algorithms are the second most widely used approaches in the primary studies. It covers about 18% of the total studies and were especially introduced in (Mostafa et al., 2017) and (). The basic idea of Coverage based approaches are to cover 100 percent coverage of the system in regression testing.

A pattern-based approach (Anderson et al., 2019) introduced the usage of fingerprint pattern based on software using telemetry. The fault-based TCP approaches were the third highest in the primary studies (Alves et al., 2016; Do et al., 2010; Jiang et al., 2012; Mei, Cai, Jia et al, 2015; Tahvili et al., 2016; Wang et al., 2015; Yu & Lau, 2012), covering almost 17% of studies. In these techniques, specific test suit may be constructed for an easy detection of the faults (Jiang et al., 2012).

Requirement based and history-based approaches are the next most widely used approaches in TCP. In (Badwal & Raperia, 2013; Chen, Zhu, Chen et al, 2018; Hettiarachchi

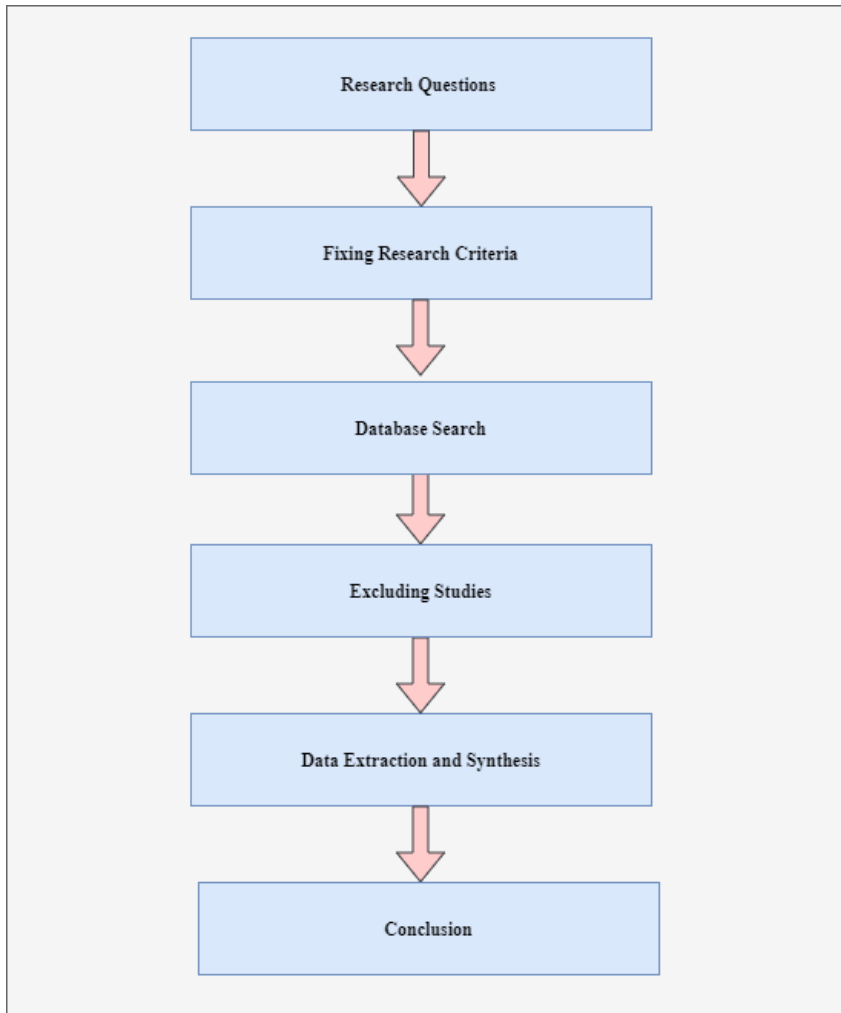
Figure 5. Distribution of research papers from year 2003 to 2020



et al., 2014; Ma et al., 2016; Muthusamy & K, 2014; Srikanth, Hettiarachchi, & Do, 2016; Yoon et al., 2012), primary studies had the requirement data for test case prioritization. Requirement data is utilized to generate test cases during buildup of the system around the requirements. History based approaches have been employed most profoundly in (Aman, 2018),(Cho et al., 2016; Huang et al., 2012; Khalilian et al., 2012; Lin et al., 2013; Marijan et al., 2013; Srikanth, Cashman, & Cohen, 2016) for test case prioritization, which works based on the previous recorded data.

Risk based approaches have been employed in (Hettiarachchi et al., 2014), (Alves, 2016; Felderer & Schieferdecker, 2014; Hettiarachchi et al., 2016; Yoon & Choi, 2011) successfully, which are further classified into two categories viz., model based and requirement-based risk approaches. In requirement-based risk approach, the risk index is computed, while in model-based approach, the

Figure 6. Classification of TCP approaches



risk is computed for every node in the model, for instance activity diagram and transition diagram etc. Test cases might be prioritized based on the calculated value of the index score.

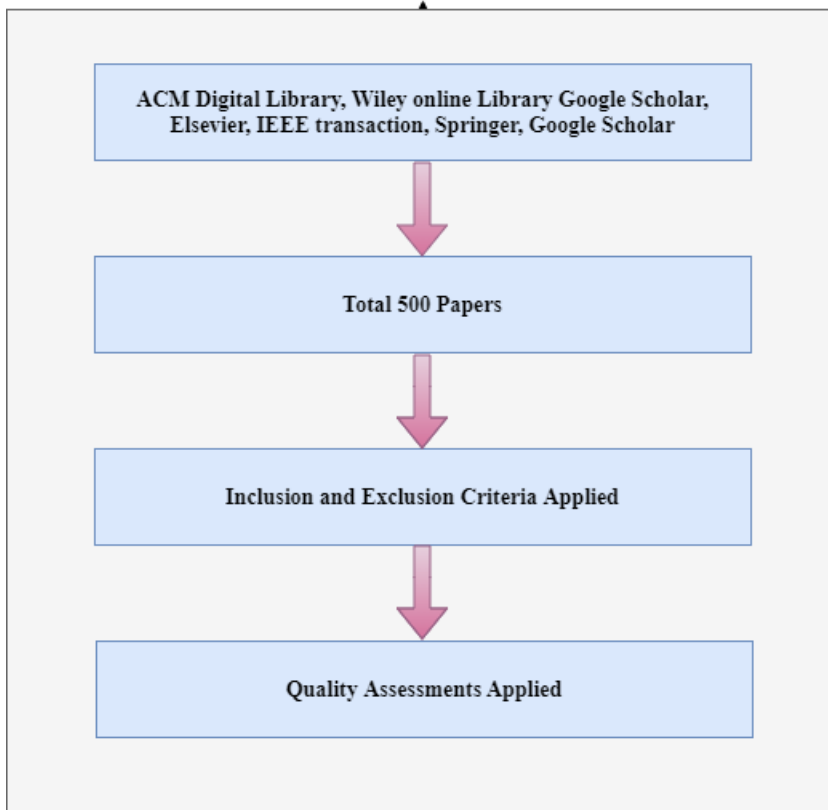
In (Mirarab & Tahvildari, 2007; Mirarab & Tahvildari, 2008; Ufuktepe & Tuglular, 2016; Zhao et al., 2015), Bayesian Network based approaches were used that had some common features like model-based approaches. Through accessing the required data, these approaches are suitable in determining the possible errors in the system (Mirarab & Tahvildari, 2008).

The cost-effective based approaches (Yu & Lau, 2012),(Huang et al., 2012),(Elbaum et al., 2004) performs the act of prioritization in view of cost incurred in software development and maintenance. Some TCP approaches works based on multi objective environment for testing web applications (Khanna & Chauhan, 2018).

4.4 Explanation of Strength and Weakness of the Available Prioritization Techniques

The description of each TCP approach is illustrated in Table 10. This overview provides the working principle of TCP approaches. The strength and weakness of each approach is listed in Table 13 of

Figure 7. Various TCP approaches



the appendix section. The analysis provided therein might be a source of motivation to researchers for pursuing potential research in near future.

4.5 Applying TCP Approaches and Effects on TCP Results (RQ-4)

To answer research question (RQ) 4, primary studies were studied against their experimental setup and the obtained results. In the subsequent section, we described about an application and impact of each approach on the TCP process.

4.5.1 Search Based Prioritization (SBP)

SBP approach has been implemented on a large scale using Genetic algorithm (GA) (), Ant Colony algorithm (ACA) (Solanki et al., 2016), (Ledru et al., 2012), Greedy (Chi et al., 2020),(Li et al., 2007), String distance (Ledru et al., 2012), and other algorithms (Eghbali & Tahvildari, 2016). The SBP study reveals that GA works poorer than Greedy approach. Based on the input criterion, fitness function, and the application of search algorithms may differ. The GA poses the main limitation in terms of execution time, which is higher than the other algorithms (i.e., Greedy and ACA etc.) reported in (Catal, 2012).

4.5.2 Coverage Based Prioritization (CBP)

CBP approaches follow the direct approach to inspect the software codes (Nardo et al., 2015),(Mahdieh, 2020). CBP approach utilizes function, statement, and branch as the main criteria to perform the

Table 10. TCP approaches: An overview

Search based	-These vary in terms of implementation. -Greedy approaches are highly used. -These are based on the fitness function of selected test case and other things like application of different AI strategies.
Coverage based	-These approaches refer to fundamental testing approaches and are able to inspect the code directly [64]. -Two main categories for code, statement, branch, or function coverage lies within total and additional coverage [10], [69].
Fault based	-This TCP approach was mentioned by E. L. G. Alves [77]. -The strategies mentioned in [78],[81] generates a sequence of test cases for fault detection.
Requirement based	-These approaches need requirements information during the requirement gathering such as error prone value, customer priority, volatility value, and execution difficulties for prioritization criteria [84], [87]
History based	-Based on the observation in the primary studies, these approaches uses history data as an input to execute test case prioritization [92].
Risk based	-Risk based approaches are mainly utilized in a project that concerns on risk values related to the software to be developed [97]. From requirement and system model, risks can be calculated for the same.
Bayesian network based	-Bayesian network-based approaches foresee the possibility of error in every single test case by the means of different accessible data [104].
Cost effective based	-This TCP technique was reported by Huang [96], [105]. -This integrates historical data to minimize the testing cost.
Topic Model based	Topic Model based techniques make use of linguistic data such as remarks, identifier markers, and string literals to support TCP functionality [22][23].

inspection of codes (Wang et al., 2017),(Hao, Zhang, Zang et al, 2016). All these criteria can be classified into two categories named as total coverage and additional coverage (Yoo & Harman, 2007). In the total coverage, the test cases are managed in such a way that these covers maximum portion of the system based on the given criteria. If the test cases cover the same area, in such case, the confliction is firstly removed by using some arbitrary method. While, in additional coverage approach, we tend to cover the complete code for the TCP test cases within stipulated time.

4.5.3 Requirement Based Prioritization (RBP)

As we know that every system is built around requirements. Requirement information is more important than code data to generate crucial test cases for software testing. Prioritization of requirements for test cases can be found in (Yoon et al., 2012) and (Hettiarachchi et al., 2014) in greater detail. Based on the score of requirement information, the TCP sequence is created. There are four factors named client priority, error prone value, volatility value and execution difficulties, based on which evaluation of test cases sequence relies. For effective test prioritization, two or more factors can be used for the RBP approaches as discussed in (Srikanth, Hettiarachchi, & Do, 2016) and (Chen, Zhu, Chen et al, 2018). In the work (Muthusamy & K, 2014), traceability, impact of fault requirements, completeness, customer priority, change in requirements and developer view has been considered to prioritize the test cases. This approach claims a better results in terms of fault detection rate.

4.5.4 Fault Based Prioritization (FBP)

In this approach, test case sequence is generated to detect the targeted faults with certainty. L.G Alves et al. (Alves et al., 2016) proposed refactoring based approach (RBA) using a set of refactoring fault

models. Specific faults can be revealed on execution of a particular statement. It might be possible that other errors could have been revealed in the same statement when the other test cases are executed. Basis on the specification, Yu, and Lau (2012) proposed Fault Adequate Test size (FATE) for determining minimal fault adequate subset. The FATE metric with lower value indicates the higher chances of recognizing the faults.

4.5.5 Risk Based Prioritization (RBP)

The project possessing the risk related values during the development phase utilizes risk-based approaches for test case prioritization (Srikanth, Hettiarachchi, & Do, 2016). In some cases, information is used to categorize crucial test cases for identifying faults in the system based on the RBP (Yoon et al., 2012),(Yoon & Choi, 2011). Hettiarachchi et al. (2014) proposed five step approaches based on TCPs requirement risk. Requirement priorities and risk values were calculated in first four steps and the prioritization was done in the final step.

4.5.6 Cost-Aware Based Prioritization (CABP)

The CABP technique was proposed by Huang et al. (2012). In their study, they employed modified cost-cognizant TCP to discern the number of faults per unit testing rate using genetic algorithm. The experiment was conducted based on flex and sed Unix utilities. In this paper, authors made a comparison between optimal, random, time aware, and historical faults. The higher values of average fault detection rate are used for prioritization, when number of generations are high pertaining to a particular software code.

4.5.7 History Based Prioritization (HBP)

In HBP techniques, history data is used for test case prioritization. Y. Cho et al. (2016) introduced an AFSAC (stands for prioritization by Accumulation of Failures with weights from Statistical Analysis and Correlation data) approach using historical data to get the failure information of the software in an effective manner. Khalilian et al. (2012) used the historical test data in their work to enhance the fault detection rate. The proposed technique shows an improvement in TCP using historical data. However, historic approaches offer some limitations like availability of historic information and availability of little information on defects, but still tends to be the part of the effective approaches.

4.5.8 Bayesian Network-Based Prioritization (BNP)

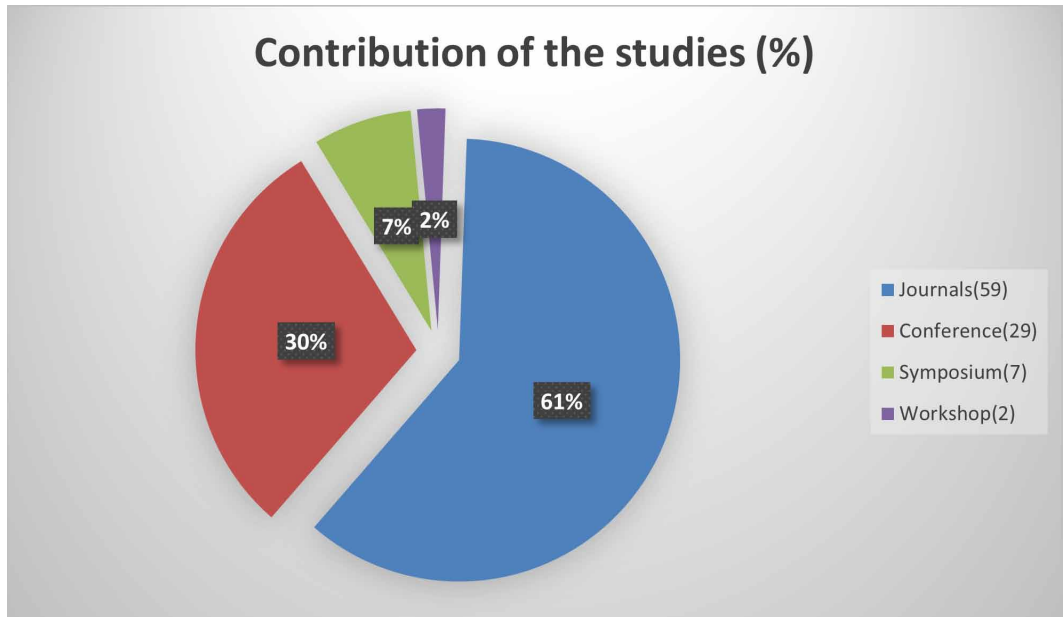
In Bayesian Network model (BNM), errors are discovered for every test case by means of different accessible data (Mirarab & Tahvildari, 2007). In the work proposed by Mirab and Tahvildari (2007), the following TCP difficulties were addressed; (i) finding different collection verification from coding portion, (ii) incorporating data in BNM, and (iii) possibility of accomplishment from probabilistic interpretation.

4.6 TCP Process (RQ 5)

Engineering processes tends to develop a software using a systematic approach that pertains to software engineering. Investigation of research question is mandatory to fulfill the desired goal. To meet the desired goal, the primary studies are examined initially to answer the research question as per the flow of the experiment. Every experiment has their own flow process that needs to be followed, while implementing the TCP process. This SLR sheds light on various processes involved in TCP. Various process works in similar manner except minor differences that takes place owing to addition of few extra steps in the process. Table 14 in appendix section display the various steps involved in the process.

Figure 8 demonstrates the process flow of prioritization techniques. As can be noticed in Fig. 8, preparation of target data is considered as the initial step of the TCP processes. The data that is

Figure 8. Flow for the TCP approaches



essential to conduct a particular experiment needs to be searched out and investigated from the utility perspective point of view. Source code, system model or requirement statements can be utilized as data for the TCP process. In the next step, data dependency can be calculated based on the data prioritization criterion. In studies conducted by Brayce et al. (2011) and Eghbali et al. (2016), the test cases were firstly determined. In Risk based techniques (Yoon et al., 2012), (Hettiarachchi et al., 2014), (Hettiarachchi et al., 2016), a risk value for a particular test case was computed and subsequently utilized to determine the prioritization. In requirement-based techniques (Srikanth, Hettiarachchi, & Do, 2016), (Ma et al., 2016), (Muthusamy & K, 2014), after following up of requirement criterion, the process of test case prioritization starts off by keeping in mind the customer's priority. In the work of Ma (2012) and Wang (2015), the TCP process iterates until all the test cases, requirement and faults got fully satisfied. Finally, results are measured in view of the complete TCP process. All the approaches have different timeline for TCP process based on the specific data information within the project schedule. Some techniques initiate TCP during or after performing the requirement analysis in software testing, while other approaches start TCP after completion of design phase.

4.7 Evaluation Metrics Used in TCP With Software Artifacts (RQ 6)

This section presents two sub sections; one sheds light on evaluation metrics, while other describes about the software artifacts. This section also demonstrates the relationship between evaluation metrics and software artifacts.

4.7.1 Software Artifacts Utilized in TCP Approaches

Artifact plays a pivotal role to conduct a controlled research for software testing and software development. Test cases, fault records, history evidence, suitability and type of research decides, what kind of artifact shall be used. Y. Sing et al. (2012) presented a careful examination of artifacts used by different researchers.

In the study (Ali et al., 2019), 50% of the artifacts are accessible without any restriction from software infrastructure repository (SIR). In our SLR, we have considered only 35% of artifacts

extracted from the SIR. For this purpose, various real case studies (Di Nardo et al., 2013), (Nardo et al., 2015), (Bryce et al., 2011), (Alves et al., 2016; Anderson et al., 2019; Krishnamoorthi & Sahaaya Arul Mary, 2009; Tahvili et al., 2016; Wang et al., 2015), (Srikanth, Hettiarachchi, & Do, 2016), (Ma et al., 2016), (Hettiarachchi et al., 2014) pertaining to different types of artifacts have been used. In general, there are the three categories of artifacts from SIR viz., Siemen, space, and TLS.

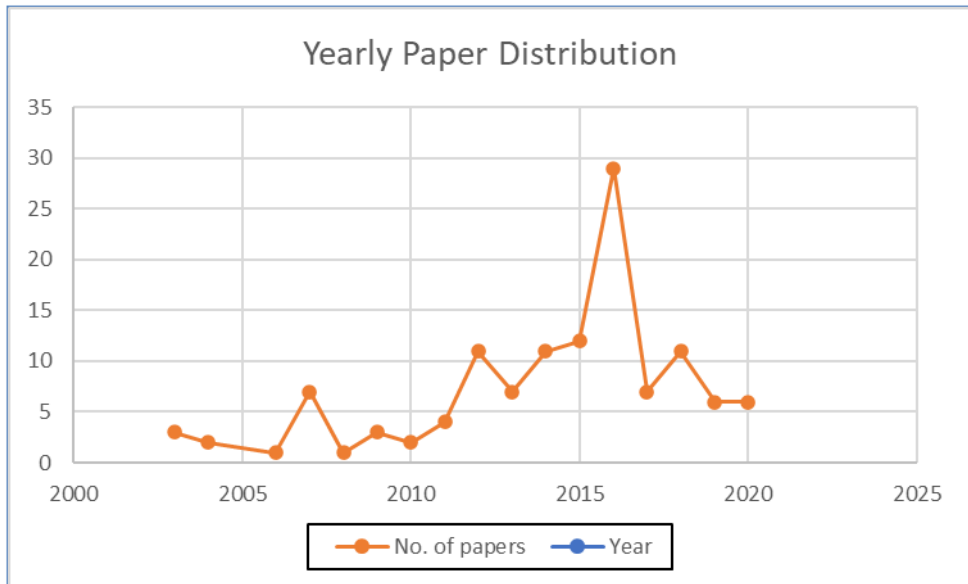
The Table 11 list out the programs run by Siemen Corporate Research, which have been accessed in literature (Singh, 2012),(Catal, 2012),(Li et al., 2007),(Ledru et al., 2012),(Hao et al., 2014),(Yoon et al., 2012) by different researchers. Test Specification Language programs are next popular category of SIR, which are cited in many research works such as (Singh, 2012),(Miranda & Bertolino, 2017),(Korel et al., 2007),(Yuan et al., 2015),(Catal, 2012),(Hao et al., 2014),(Jiang et al., 2012),(Huang et al., 2012), and (Elbaum et al., 2004). The studies (Khanna & Chauhan, 2018) and (Sujata & Purohit, 2017) rely upon the TCP approaches and their work mainly focus on multiple objectives. The test case selection, minimization and prioritization survey done by M. Harman (2012) provides a wide variety of approaches for all dimensions of regression testing. The study (Sanchez et al., 2014) presents a comparison of TCP approaches for various types of software. A study done by Nasir M. Minhas et al. (2020) explored the state of practices based on interviews conducted with an embedded software development team. Requirement based studies involving risk factors are presented in considerable detail in Ref. (Arafeen & Do, 2013), while the studies (Singh, 2012) and (Kapfhammer & Soffa, 2007) are based on random inputs and cost effectiveness. A study (Yadav & Dutta, 2016) conducted by S. Dutta et. al. presented an early fault detection technique based on fuzzy logic for TCPs. The Unix utilities flex, grep, gzip and make are under the TSL programs, while Space is the third SIR (Software-artifact Infrastructure Repository, n.d.) with 6218 executable lines of code, which have been utilized in various studies cited as (Singh, 2012),(Yuan et al., 2015)[43],(Do & Rothermel, 2006),(Li et al., 2007),(Hao et al., 2014),(Hao, Zhang, Zang et al, 2016),(Khalilian et al., 2012). The space program is used for validation of a file and act like an interpreter. The TCP approaches that do not intend to use the requirement-based approach adopt the SIR programs. Figure 9 demonstrate the TCP approaches and SIR programs in percentage.

Fig. 10 shows that 67% and 16% of the SIR is utilized in search-based and fault-based TCP research (),(Do et al., 2010; Jiang et al., 2012; Mei, Cai, Jia et al, 2015; Wang et al., 2015; Yu & Lau, 2012),(Yadav & Dutta, 2016), respectively. The study (Schwartz & Do, 2016) presents an adaptive and cost-effective TCP approach. Some studies (Fu et al., 2016; Khanna et al., 2019; Laali et al., 2016; Marchetto et al., 2016; Xia et al., 2016) presents fault based, fault localization based and multi objective based approach. Other approaches like coverage and history also utilize the SIR programs to a little extent. SIR is used by the above approaches mainly due to the artifacts that provides in terms of source code, test suit, fault information, coverage information and history evidence. However, requirements-based approaches do not utilize it due to not being provided in SIR.

Table 11. Siemen programs

S. No.	Program	Description (LOC)
1	REPLACE	Pattern Replace (564)
2	PRINTTOKENS	Lexical Analyzer (726)
3	PRINTTOKENS2	Lexical Analyzer (526)
4	TOT_INFO	Information Statistic Measure (565)
5	SCHEDULE	Priority Scheduler (412)
6	SCHEDULE2	Priority Scheduler (374)
7	TCAS	Aircraft Collision Avoidance (173)

Figure 9. Category wise usages of SIR



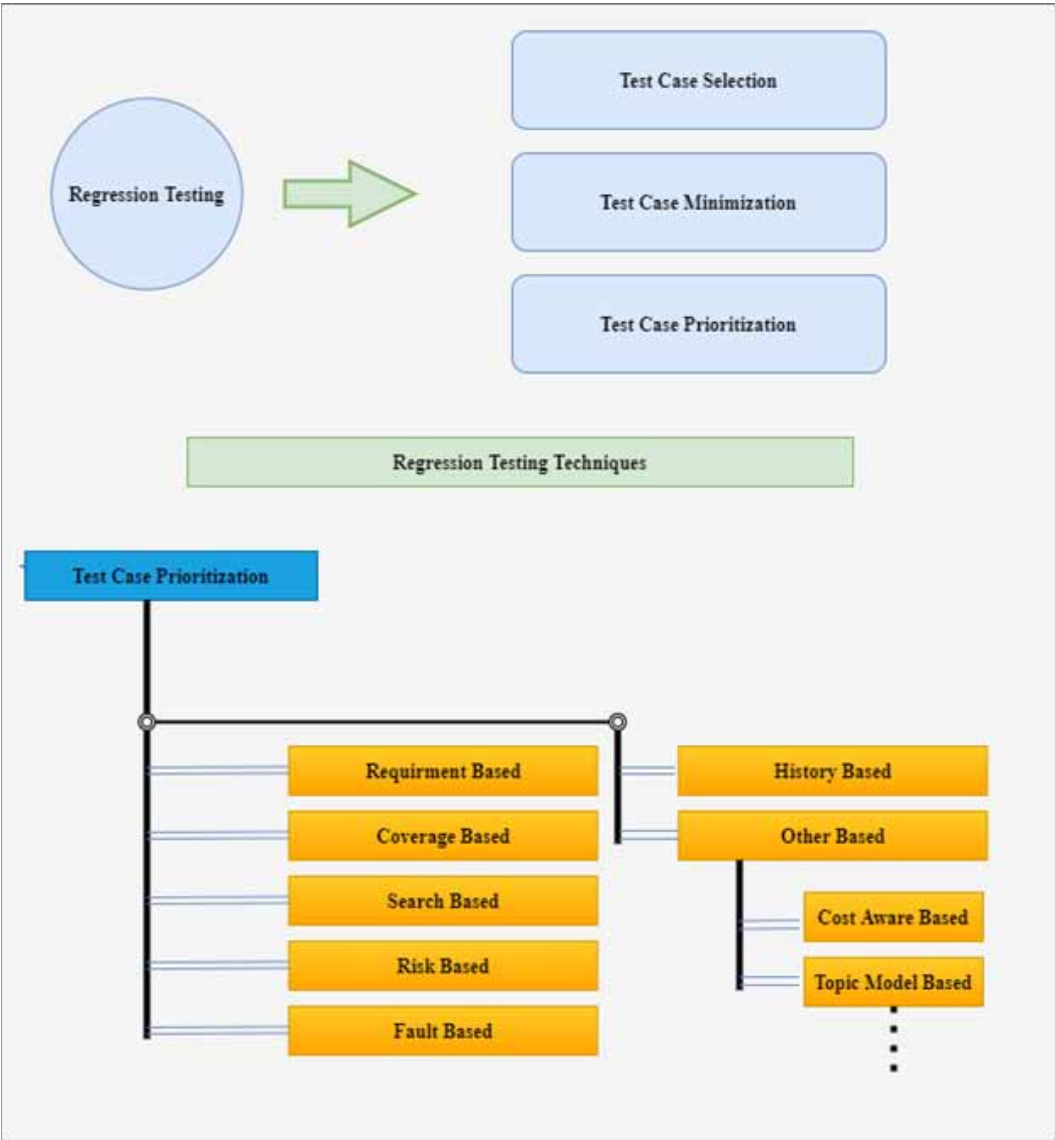
4.7.2 Evaluation Metric for TCP Approaches

A performance metrics is used for performance evaluation of any approach. Evaluation metric is used to measure efficiency of a TCP approach and to benchmark its effectiveness against other approaches. The Fig. 10 presents various metrics that have been widely utilized in literature. The Fig. given below shows that APFD is most popular performance metrics that accounts 50% of the total metrics used. Others, time, CE, and APFDc accounts for remaining 50% of the performance metrics.

The APFD and coverage effectiveness metric accounts for 9%, and 10%, respectively with regards to cost. The time execution and others have the proportion of 8% and 22%, respectively. The APFD metric has been widely utilized by (Thomas et al., 2012), (Sampath et al., 2013), (Mei, Chan, Tse et al, 2015), (), (Solanki et al., 2016), (Gao et al., 2015), (Ledru et al., 2012), (), (Ma et al., 2016), (Muthusamy & K, 2014), (Hettiarachchi et al., 2014), (Hettiarachchi et al., 2016), (Alves, 2016), (Yoon & Choi, 2011), and (Arafeen & Do, 2013; Elbaum et al., 2004; Henard, 2016; Jiang & Chan, 2015; Khanna & Chauhan, 2018; Minhas, 2020; Sujata & Purohit, 2017; Yoo & Harman, 2012). APFD quantify that how fast an optimized test suit can discover defects. The results of APFD approach are presented quantitatively between 0 to 100. Higher value indicate better fault detection rate. To fulfill different objective of prioritization, other factors such as time and cost are also included as an effective performance metrics.

Second most utilized approach is coverage effectiveness that have been utilized by (Thomas et al., 2012), (Mei, Chan, Tse et al, 2015), (Miranda & Bertolino, 2017), (Korel et al., 2007), (Catal, 2012), (Ledru et al., 2012), (Fang et al., 2012), (Bryce et al., 2011), and (Jiang et al., 2012; Mei, Cai, Jia et al, 2015; Wang et al., 2015). This metric pertains to the size of test suit and the coverage of each test case. Ratio of the size of whole test suit and coverage of re-ordered test suit is known as coverage effectiveness (CE) as given in (Kapfhammer & Soffa, 2007). The value of CE lies from zero to one. The higher value of CE indicates better effectiveness. APFDc has a total contribution of 9% including cost factor. In numerous studies (Kumar & Singh, 2014), (Miranda & Bertolino, 2017), (Catal, 2012), (Wang et al., 2015), (Marijan et al., 2013), (Lin et al., 2013), (Huang et al., 2012), (Elbaum et al., 2004), over fault detection rate is utilized to measure cost. This metric is utilized by history based and fault-based approaches too.

Figure 10. Evaluation Metrics for TCP approaches



Time execution metric is used in research (Thomas et al., 2012), (Maheswari & Mala, 2015), (Catal, 2012), (Solanki et al., 2016), (Gao et al., 2015), (Ledru et al., 2012), (Jiang et al., 2012), (Jiang & Chan, 2015) and has the contribution of 8% as shown in Fig.10. To minimize the time in TCP, this metric is used in search-based approaches to evaluate the effectiveness of the TCP approach. Other approaches contribute almost 22% of the entire distribution and are mainly based on APFD. APFD approach is also known by various names like APSC (average percentage statement coverage), APDC (average percentage decision coverage), APBC (average percentage branch coverage), TSFD (Average Percentage Statement Coverage), WPF, NAPFD and ASFD.

There is another test known as ANOVA, which is a popular statistical method given by Fisher. This method works on the evaluation of difference of means of three or more groups and have been utilized in previous studies (Thomas et al., 2012), (Fang et al., 2014), (Ledru et al., 2012), (Di

Nardo et al., 2013), (Hao et al., 2014), (Zhang et al., 2013), (Jones & Harrold, 2003a), (Alves et al., 2016), (Srikanth, Hettiarachchi, & Do, 2016), (Zhao et al., 2015), (Elbaum et al., 2004; Mirarab & Tahvildari, 2008) on large scale. For any approach under evaluation, ANOVA test provides the statistical significance in terms of p value.

5. RESEARCH FINDINGS

Prioritizing the test cases is an important activity in testing phases (Schwartz & Do, 2016). At the time of resuming the project, client expectations are high, while delivery time is short with the limited assets. In addition to this, the software must have fewer fault fulfilling all the requirements. TCP techniques are very crucial from the perspective of cost, time and software faults as TCP tend to reorder the test cases for early fault detection. In addition to testcase optimization, TCP is also employed for early delivery of software. The impact of TCP is prominent in regression testing.

All research studies are supposed to answer RQ1. From this, it can be inferred that TCP approaches are open for further improvements. Every year, new approaches are evolving for test case prioritization.

To get the answer of RQ2, recently numerous techniques are available on latest trends such as artificial intelligence and machine learning. These approaches have their own advantages and limitations. For instance, two or more criteria can be tackled in multi-objective approaches (Marchetto et al., 2016), (Khanna et al., 2019). This approach can be mingled with other approaches that makes it more efficient. From the above discussion, we can conclude that the TCP is an important component of regression testing as it reduces the time, cost, and fault detection rate.

RQ3 and RQ4 belongs to positive and negatives of the approaches. These questions mainly focus on the adopted methodology and effectiveness of a particular approach. In this SLR, out of total studies, 40% of the studies were considered for examination to answer these questions. Each approach has its own advantage, limitations, and potential values. On the basis of data set used, requirement-based approaches take the input in terms of customer priority for generating the test cases. In risk-based approaches, the risk value is considered as an input for prioritization process. This shows that both approaches have specific advantages in comparison to other techniques in terms of initial execution point of TCP. It is because both the approaches resume before the code starts. The strengths and limitations of TCP function and services motivates us for making amendments in software.

To answer RQ5, total 30 studies were examined against the experimental setup. In certain projects, the TCP plays a crucial role. The project schedule can be adjusted by using TCP techniques and project managers may get benefitted by this. Owing to the variation in initial process of TCP of different approaches, project managers get different benefits and timeline to complete the projects as per the suitability of the project schedule.

To answer RQ6, all the primary studies were considered for full examination. In these, almost all the metrics were utilized into their entirety. These approaches used APFD metrics in almost, all the TCP approaches. To support different objective in different approaches, some more metrics were also adopted from APFD. It is evident that programs from the SIR are also widely used by the researchers during their research work.

6. ESTABLISHING THE VALIDITY OF EXPERIMENTAL STUDIES

Non-relevant collection of research papers, unspecified data synthesis and derivation are possible threats to this SLR. This section presents collection of primary studies, estimated data synthesis, TCS, TCM, TCP and fault localization.

6.1 Collection of Primary Studies

The collection of primary studies has been elaborated in detail in section 3. It is not possible to give 100% guarantee that we have gone through all the available reviews related to prioritization for the

purpose of answering all the research questions. Search string is one of the important reasons of threat as some paper might not have been covered by our search strings. However, the authors have used various search strings to extract most of the studies, which have been included in this SLR, the details of which are given in section 3.4 in detail. Apart from this, there are number of studies, which might require different keywords than those, which have been used for this review. Another issue is related to non-English language papers. It is because some TCP studies might be available in other languages too.

6.2. Estimated Data Synthesis and Retrieval

Estimated data synthesis and incomplete extraction process may be the next threat to validity of this SLR. This threat may be due to the invalid data classification being not following a systematic approach for data extraction. This problem can be easily resolved, if extraction of non-relevant data is minimized by using manual scrutiny procedure. Error related to data collection could be reduced, if we consider selected studies only as discussed in sec 3.6. In our SLR, inaccurate studies were removed by applying some quality assessments on the candidate studies.

6.3. TCS, TCM, TCP and Fault Localization

There are the following three terms, which have been used quite often in regression testing: (i) test case selection (TCS), (ii) test case minimization (TCM), and (iii) test case prioritization (TCP). TCS is used to select related test cases and executes the whole test suit by localizing fault. Hence, TCS is analyzed like TCP. In this study TCP does not consider current trends related to fault localization and this may be a possible threat to the validity of this SLR. In numerous works available in (Fu et al., 2016; Xia et al., 2016), the fault localization tends to maximize the testing speed. After prioritization, the TCP approach can be utilized in TCS owing to the selection of test cases, which are gained based on the prioritization outcomes. In the study (Xia et al., 2016), an approach based on diversity maximization speed up is proposed. By applying fault localization, this approach ranks up the test cases related to a particular program unit. This technique can achieve the desired goal in terms of the minimum cost.

7. CONCLUSION

The purpose of this study was to discuss and classify the current trends and the state of affairs in TCP techniques. Here, we conducted literature review in systematic way to get aware of the current trends and advancements in TCP. According to the scheme adopted in this SLR, we examined and classified the TCP approaches based on the mapping with six research questions that had higher relevance to the TCP approaches. In this study, we systematically examined the primary studies to discern the scope of improvement and bridge the gap in currently available TCP approaches.

Furthermore, the results and comprehensive data are presented as figures and tables to get better understanding of research work done by other researchers. Their work can be briefly summarized as follows:

1. However, there is a great number of prioritization techniques available, but room for the improvements is still available.
2. Requirement statement, source code and system models in TCP requires extensive research to be conducted.
3. For each approach, time frame for execution of TCP process needs to be completely known in advance.
4. In system development life cycle, early prioritization plays a pivotal role, therefore has a great scope in research. In this regard, the project manager and development team can adjust various schedule and timeline.

5. To reduce human error, computerized and automated reasoning needs to be adopted instead of human driven decision making for estimation of cost and time.
6. More we rely on the automated process for making decisions regarding test case execution, higher will be the chances to finish the assignment on the scheduled time.
7. In next research work, we will primarily focus on how the fault localization can help TCP in comparison to test case selection (TCS).

REFERENCES

- Achimugu, P., Selamat, A., Ibrahim, R., & Naz, M. (2014). A systematic literature review of software requirements prioritization research. *Information and Software*, 56(6), 568–585. doi:10.1016/j.infsof.2014.02.001
- Ahmad, S. F., Singh, D. K., & Suman, P. (2018). Prioritization for Regression Testing Using Ant Colony Optimization Based on Test Factors. In *Advances in Intelligent Systems and Computing* (Vol. 624). Springer.
- Ali, N., Engström, E., Taromirad, M., Mousavi, M. R., Minhas, N. M., Helgesson, D., Kunze, S., & Varshosaz, M. (2019). On the search for industry-relevant regression testing research. *Empirical Software Engineering*, 24(4), 2020–2055. doi:10.1007/s10664-018-9670-1
- Alves, , E. L. G. (2016). *Prioritizing Test Cases for Early Detection of Refactoring Faults*. Software Testing, Verification and Reliability.
- Alves, E. L. G., Machado, P. D. L., Massoni, T., & Kim, M. (2016). Prioritizing Test cases for early detection of refactoring faults. *Software Testing, Verification & Reliability*, 26(5), 402–426. doi:10.1002/stvr.1603
- Aman, H. (2018). A Topic Model and Test History-Based Test Case Recommendation Method for Regression Testing. *IEEE International Conference on Software Testing, Verification and Validation Workshops*, 392-397. doi:10.1109/ICSTW.2018.00079
- Anderson, J., Azizi, M., Salem, S., & Do, H. (2019). On the use of usage patterns from telemetry data for test case prioritization. *Information and Software Technology*, 113, 110–130. doi:10.1016/j.infsof.2019.05.008
- Anuradha, C., & Neha, N. (2018). Application of Evolutionary Particle Swarm Optimization Algorithm in Test Suite Prioritization. In *Computational Vision and Bio Inspired Computing. Lecture Notes in Computational Vision and Biomechanics* (vol. 28). Springer. doi:10.1007/978-3-319-71767-8_2
- Arafeen, M. J., & Do, H. (2013). Test Case Prioritization Using Requirements-Based Clustering. *2013 IEEE Sixth International Conference on Software Testing, Verification and Validation*, 312–321. doi:10.1109/ICST.2013.12
- Azizi, M., & Do, H. (2018). Graphite: A Greedy Graph-Based Technique for Regression Test Case Prioritization. *2018 IEEE International Symposium on Software Reliability Engineering Workshops [ISSREW]*, 245-251. doi:10.1109/ISSREW.2018.00014
- Badwal, J., & Raperia, H. (2013). Test Case Prioritization using Clustering. *IEEE Sixth International Conference*, 488–492.
- Banias, O. (2019). Test case selection-prioritization approach based on memoization dynamic programming algorithm. *Information and Software Technology*, 115, 119–130. doi:10.1016/j.infsof.2019.06.001
- Bian, Y., Li, Z., Zhao, R., & Gong, D. (2017). Epistasis Based ACO for Regression Test Case Prioritization. *IEEE Transactions on Emerging Topics in Computational Intelligence*, 1(3), 213-223. doi:10.1109/TETCI.2017.2699228
- Bryce, R. C., Sampath, S., Pedersen, J. B., & Manchester, S. (2011). Test suite prioritization by cost-based combinatorial interaction coverage. *International Journal of Systems Assurance Engineering and Management*, 2(2), 126–134. doi:10.1007/s13198-011-0067-4
- Catal, C. (2012). On the application of genetic algorithms for test case prioritization: a systematic literature review. *Proceedings of the 2nd International Workshop on Evidential assessment of software technologies*, 9-14. doi:10.1145/2372233.2372238
- Catal, C., & Mishra, D. (2012). Test case prioritization: A systematic mapping study. *Software Quality Journal*, 21(3), 445–478. doi:10.1007/s11219-012-9181-z
- Chen, J., Lou, Y., Zhang, L., & Zhou, J. (2018). Optimizing Test Prioritization via Test Distribution Analysis. *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 656–667. doi:10.1145/3236024.3236053
- Chen, J., Zhu, L., Chen, T. Y., Towey, D., Kuo, F.-C., Huang, R., & Guo, Y. (2018). Test case prioritization for object-oriented software: An adaptive random sequence approach based on clustering. *Journal of Systems and Software*, 135, 107–125. doi:10.1016/j.jss.2017.09.031

- Chi, Qu, Zheng, Yang, Jin, Cui, & Liu. (2020). Relation-based test case prioritization for regression testing. *Journal of Systems and Software*, 163.
- Chittimalli, P. K., & Harrold, M. J. (2009). Recomputing coverage information to assist regression testing. *IEEE Transactions on Software Engineering*, 35(4), 452–469. doi:10.1109/TSE.2009.4
- Cho, Y., Kim, J., & Lee, E. (2016). History-Based Test Case Prioritization for Failure Information. *2016 23rd Asia-Pacific Software Engineering Conference*, 385–388. doi:10.1109/APSEC.2016.066
- Di Nardo, D., Alshahwan, N., Briand, L., & Labiche, Y. (2013). Coverage-based test case prioritization: An industrial case study. *Proceedings - IEEE 6th International Conference on Software Testing, Verification and Validation, ICST 2013*, 302–311.
- Di Nucci, D., Panichella, A., Zaidman, A., & De Lucia, A. (2020). A Test Case Prioritization Genetic Algorithm Guided by the Hypervolume Indicator. *IEEE Transactions on Software Engineering*, 46(6), 674–696. doi:10.1109/TSE.2018.2868082
- Do, H. (2016). Recent Advances in Regression Testing Techniques. *Advances in Computers*, 103, 53–77. doi:10.1016/bs.adcom.2016.04.004
- Do, H., Mirarab, S., Tahvildari, L., & Rothermel, G. (2010). The Effects of Time Constraints on Test Case Prioritization: A Series of Controlled Experiments. *IEEE Transactions on Software Engineering*, 36(5), 593–617. doi:10.1109/TSE.2010.58
- Do, H., & Rothermel, G. (2006). On the use of mutation faults in empirical assessments of test case prioritization techniques. *IEEE Transactions on Software Engineering*, 32(9), 733–752. doi:10.1109/TSE.2006.92
- Eghbali, S., & Tahvildari, L. (2016, December 1). Test Case Prioritization Using Lexicographical Ordering. *IEEE Transactions on Software Engineering*, 42(12), 1178–1195. doi:10.1109/TSE.2016.2550441
- Elbaum, S., Rothermel, G., Kanduri, S., & Malishevsky, A. G. (2004). Selecting a cost-effective test case prioritization technique. *Software Quality Journal*, 12(3), 185–210. doi:10.1023/B:SQJO.0000034708.84524.22
- Elbaum, S., Rothermel, G., & Penix, J. (2014). Techniques for improving regression testing in continuous integration development environments. *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering - FSE*, 235–245. doi:10.1145/2635868.2635910
- Fang, C., Chen, Z., Wu, K., & Zhao, Z. (2014). Similarity-based test case prioritization using ordered sequences of program entities. *Software Quality Journal*, 22(2), 335–361. doi:10.1007/s11219-013-9224-0
- Fang, C. R., Chen, Z. Y., & Xu, B. W. (2012). Comparing logic coverage criteria on test case prioritization. *Science China. Information Sciences*, 55(12), 2826–2840. doi:10.1007/s11432-012-4746-9
- Felderer, M., & Schieferdecker, I. (2014). A taxonomy of risk-based testing. *International Journal of Software Tools for Technology Transfer*, 16(5), 559–568. doi:10.1007/s10009-014-0332-3
- Flemstrom, D., Potena, P., Sundmark, D., Afzal, W., & Bohlin, M. (2018). Similarity-based prioritization of test case automation. *Software Quality Journal*, 26(4), 1421–1449. doi:10.1007/s11219-017-9401-7
- Fu, W., Yu, H., Fan, G., & Ji, X. (2016). Test Case Prioritization Approach to Improving the Effectiveness of Fault Localization. *International Conference on Software Analysis, Testing and Evolution [SATE]*, 60–65. doi:10.1109/SATE.2016.17
- Gao, D., Guo, X., & Zhao, L. (2015). *Test case prioritization for regression testing based on ant colony optimization*. *Software Engineering and Services*. doi:10.1109/ICSESS.2015.7339054
- Hao, Zhang, Zhang, Rothermel, & Mei. (2014). A Unified Test Case Prioritization Approach. *ACM Trans Softw Eng Methodol*, 24(2), 10:1–10:31.
- Hao, D., Zhang, L., & Mei, H. (2016). Test-case prioritization: Achievements and challenges. *Frontiers of Computer Science*, 10(5), 769–777. doi:10.1007/s11704-016-6112-3
- Hao, D., Zhang, L., Zang, L., Wang, Y., Wu, X., & Xie, T. (2016). To Be Optimal or Not in Test-Case Prioritization. *IEEE Transactions on Software Engineering*, 42(5), 490–505. doi:10.1109/TSE.2015.2496939

Henard, C. (2016). Comparing White-box and Black-box Test Prioritization. *Proceedings of the 38th International Conference on Software Engineering*, 523–534.

Hettiarachchi, C., Do, H., & Choi, B. (2014). Effective regression testing using requirements and risks. *Proceedings - 8th International Conference on Software Security and Reliability, SERE 2014*, 157–166. doi:10.1109/SERE.2014.29

Hettiarachchi, C., Do, H., & Choi, B. (2016). Risk-based test case prioritization using a fuzzy expert system. *Information and Software Technology*, 69, 374–385. doi:10.1016/j.infsof.2015.08.008

Huang, R., Zhou, Y., Zong, W., Towey, D., & Chen, J. (2017). An Empirical Comparison of Similarity Measures for Abstract Test Case Prioritization. *IEEE 41st Annual Computer Software and Applications Conference*, 3–12. doi:10.1109/COMPSAC.2017.271

Huang, Y. C., Peng, K. L., & Huang, C. Y. (2012). A history-based cost-cognizant test case prioritization technique in regression testing. *Journal of Systems and Software*, 85(3), 626–637. doi:10.1016/j.jss.2011.09.063

Jackson, A. (2020). *Test Case Prioritization in Continuous Integration Environments: A Systematic Mapping Study* (Vol. 121). Information and Software Technology.

Jeffrey, D., & Gupta, N. (2007, February). Improving fault detection capability by selectively retaining test cases during test suite reduction. *IEEE Transactions on Software Engineering*, 33(2), 108–123. doi:10.1109/TSE.2007.18

Jiang, B., & Chan, W. (2015). Input-based adaptive randomized test case prioritization: A local beam search approach. *Journal of Systems and Software*, 105, 91–106. doi:10.1016/j.jss.2015.03.066

Jiang, B., Zhang, Z., Chan, W. K., Tse, T. H., & Chen, T. Y. (2012). How well does test case prioritization integrate with statistical fault localization? *Information and Software Technology*, 54(7), 739–758. doi:10.1016/j.infsof.2012.01.006

Jones, J., & Harrold, M. (2003a). Test-Suite Reduction and Prioritization for Modified Condition / Decision Coverage Georgia Institute of Technology. *Test*, 3(3), 101–195.

Jones, J. A., & Harrold, M. J. (2003b, March). Test-suite reduction and prioritization for modified condition/ decision coverage. *IEEE Transactions on Software Engineering*, 29(3), 195–209. doi:10.1109/TSE.2003.1183927

Jun, W., Yan, Z., & Chen, J. (2011). *Test case prioritization technique based on genetic algorithm*. Internet Computing & Information. doi:10.1109/ICICIS.2011.50

Kapfhammer, G., & Soffa, M. (2007). Using coverage effectiveness to evaluate test suite prioritizations. *Proceedings of the 1st ACM international*, 19–20. doi:10.1145/1353673.1353677

Kaur, A., & Goyal, S. (2011). A genetic algorithm for fault-based regression test case prioritization. *International Journal of Computers and Applications*, 32(8), 975–8887.

Khalilian, Azgomi, & Fazlalizadeh. (2012). An improved method for test case prioritization by incorporating historical test case data. *Science of Computer*.

Khanna & Chauhan. (2018). Search for Prioritized Test Cases in Multi-Objective Environment During Web Application Testing. *Arab J Sci Eng*, 43, 4179–4201.

Khanna, M., Chaudhary, A., Toofani, A., & Pawar, A. (2019). Performance Comparison of Multi-objective Algorithms for Test Case Prioritization During Web Application Testing. *Arabian Journal for Science and Engineering*, 44(11), 9599–9625. doi:10.1007/s13369-019-03817-7

Khanna, M., Chauhan, N., Sharma, D., Toofani, A., & Chaudhary, A. (2018). Search for Prioritized Test Cases in Multi-Objective Environment During Web Application Testing. *Arabian Journal for Science and Engineering*, 43(8), 4179–4201. doi:10.1007/s13369-017-2830-6

Khatibsyarbini, M., Isa, M. A., Jawawi, D. N. A., & Tumeng, R. (2018). Test case prioritization approaches in regression testing: A systematic literature review. *Information and Software Technology*, 93, 74–93. doi:10.1016/j.infsof.2017.08.014

- Kiran, P., & Chandraprakash, K. (2015). A Literature Survey on TCP-Test Case Prioritization using the RTRegression Techniques. *Global Journal of Researches in Engineering: Industrial Engineering*, 15(1).
- Kitchenham, B. (2004). Procedures for performing systematic reviews. Keele University.
- Kitchenham, B., & Brereton, O. P. (2009). Systematic literature reviews in software engineering -A systematic literature review. *Information and Software Technology*, 51(1), 7–15. doi:10.1016/j.infsof.2008.09.009
- Korel, B., Koutsogiannakis, G., & Tahat, L. H. (2007). Model-based test prioritization heuristic methods and their evaluation. *Proceedings of the 3rd international workshop on Advances in model-based testing*, 34–43. doi:10.1145/1291535.1291539
- Krishnamoorthi, R., & Sahaaya Arul Mary, S. A. (2009). Factor oriented requirement coverage-based system test case prioritization of new and regression test cases. *Information and Software Technology*, 51(4), 799–808. doi:10.1016/j.infsof.2008.08.007
- Kumar, A., & Singh, K. (2014). *A Literature Survey on test case prioritization*. Compusoft.
- Laali, M., Liu, H., Hamilton, M., Spichkova, M., & Schmidt, H. W. (2016). *Test Case Prioritization Using Online Fault Detection Information*. Springer. doi:10.1007/978-3-319-39083-3_6
- Ledru, Y., Petrenko, A., Boroday, S., & Mandran, N. (2012). Prioritizing Test cases with string distances. *Automated Software Engineering*, 19(1), 65–95. doi:10.1007/s10515-011-0093-0
- Li, Z., Harman, M., & Hierons, R. M. (2007, April). Search Algorithms for Regression Test Case Prioritization. *IEEE Transactions on Software Engineering*, 33(4), 225–237. doi:10.1109/TSE.2007.38
- Liang, J., Elbaum, S., & Rothermel, G. (2018). Redefining prioritization: Continuous Prioritization for continuous Integration. *Proceedings of ACM/IEEE 40th International conference on Software Engineering*. doi:10.1145/3180155.3180213
- Lin, C. T., Chen, C. D., Tsai, C. S., & Kapfhammer, G. M. (2013). History-based test case prioritization with software version awareness. *Proceedings of the IEEE International Conference on Engineering of Complex Computer Systems, ICECCS*, 171–172. doi:10.1109/ICECCS.2013.33
- Lou, Y., Chen, J., Zhang, L., & Hao, D. (2019). A Survey on Regression Test-Case Prioritization. *Advances in Computers*, 113, 1–46. doi:10.1016/bs.adcom.2018.10.001
- Lou, Y., Hao, D., & Zhang, L. (2016). Mutation-based test-case prioritization in software evolution. *2015 IEEE 26th International Symposium on Software Reliability Engineering, ISSRE 2015*, 46–57.
- Ma, T., Zeng, H., & Wang, X. (2016). Test case prioritization based on requirement correlations. *2016 IEEE/ACIS 17th International Conference on Software Engineering, Artificial Intelligence, Networking, and Parallel/Distributed Computing*, 419–424. doi:10.1109/SNPD.2016.7515934
- Mahdiah, M. (2020). *Incorporating fault-proneness estimations into coverage-based test case prioritization methods* (Vol. 121). Information and Software Technology.
- Maheswari, R., & Mala, D. (2015). Combined Genetic and -Simulated Annealing Approach for Test Case Prioritization. *Indian Journal of Science and Technology*, 8(35). Advance online publication. doi:10.17485/ijst/2015/v8i35/81102
- Marchetto, A., Islam, M., Asghar, W., Susi, A., & Scanniello, G. (2016). A multi-objective technique to prioritize test cases. *IEEE Transactions*, 42(10), 918–940. doi:10.1109/TSE.2015.2510633
- Marijan, D., Gotlieb, A., & Sen, S. (2013). *Test case prioritization for continuous regression testing: An industrial case study*. Software Maintenance.
- Mei, L., Cai, Y., Jia, C., Jiang, B., Chan, W. K., Zhang, Z., & Tse, T. H. (2015). A Subsumption Hierarchy of Test Case Prioritization for Composite Services. *IEEE Transactions on Services Computing*, 8(5), 658–673. doi:10.1109/TSC.2014.2331683
- Mei, L., Chan, W. K., Tse, T. H., Member, S., Jiang, B., & Zhai, K. (2015). Preemptive Regression Testing of Workflow-Based Web Services. *IEEE Transactions on Services Computing*, 8(5), 740–754. doi:10.1109/TSC.2014.2322621

Minhas. (2020). *Regression testing for large-scale embedded software development – Exploring the state of practice* (Vol. 120). Information and Software Technology.

Miranda, B., & Bertolino, A. (2017). Scope-aided test prioritization, selection and minimization for software reuse. *Journal of Systems and Software*, 131, 528–549. doi:10.1016/j.jss.2016.06.058

Mirarab, S., & Tahvildari, L. (2007). A Prioritization Approach for Software Test Cases Based on Bayesian Networks. *Fundamental Approaches to Software Engineering Springer Berlin Heidelberg*, 4422, 276–290. doi:10.1007/978-3-540-71289-3_22

Mirarab, S., & Tahvildari, L. (2008). An Empirical Study on Bayesian Network-based Approach for Test Case Prioritization. *1st International Conference on Software Testing, Verification, and Validation*, 278–287. doi:10.1109/ICST.2008.57

Mohd-Shafie, M. L., Wan-Kadir, W. M. N., Khatibsyarhini, M., & Isa, M. A. (2020). Model-based test case prioritization using selective and even-spread count-based methods with scrutinized ordering criterion. *PLoS One*, 15(2), e0229312. doi:10.1371/journal.pone.0229312 PMID:32084232

Mostafa, Wang, & Xie. (2017). PerfRanker: Prioritization of Performance Regression Tests for Collection-Intensive Software. *Proceedings of 26th International Symposium on Software Testing and Analysis*, 23–34.

Muthusamy, T., & K, S. (2014). A New Effective Test Case Prioritization for Regression Testing based on Prioritization Algorithm. *International Journal of Applied Information Systems*, 6(7), 21–26. doi:10.5120/ijais14-451081

Nardo, D., Alshahwan, N., & Briand, L. (2015). Coverage- based regression test case selection, minimization, and prioritization: A case study on an industrial system. *Software Testing*, 25(4), 371–396. doi:10.1002/stvr.1572

Noguchi, T., Washizaki, H., Fukazawa, Y., Sato, A., & Ota, K. (2015). History-Based Test Case Prioritization for Black Box Testing Using Ant Colony Optimization. *IEEE 8th International Conference on Software Testing, Verification and Validation*, 1–2. doi:10.1109/ICST.2015.7102622

Ralph, P. (2016). Software engineering process theory: A multi-method comparison of Sensemaking-Coevolution-Implementation Theory and function-behavior-structure theory. *Information and Software Technology*, 70, 232–250. doi:10.1016/j.infsof.2015.06.010

Sabharwal, Sibal, & Sharma. (2010). Prioritization of test case scenarios derived from activity diagram using genetic algorithm. *2010 International Conference on Computer and Communication Technology, ICCCT-2010*, 481–485.

Sampath, S., Bryce, R., & Memon, A. M. (2013). A uniform representation of hybrid criteria for regression testing. *IEEE Transactions on Software Engineering*, 39(10), 1326–1344. doi:10.1109/TSE.2013.16

Sanchez, A. B., Segura, S., & Ruiz-Cortes, A. (2014). A Comparison of Test Case Prioritization Criteria for Software Product Lines. *Software Testing, Verification and Validation, 2014 IEEE Seventh International Conference*, 41–50. doi:10.1109/ICST.2014.15

Schwartz, A., & Do, H. (2016). Cost-effective regression testing through Adaptive Test Prioritization strategies. *Journal of Systems and Software*, 115, 61–81. doi:10.1016/j.jss.2016.01.018

Singh, Y. (2012). Systematic Literature Review on Regression Test Prioritization Techniques Difference between Literature Review and Systematic Literature. *Informatica (Vilnius)*, 36, 379–408.

Software-artifact Infrastructure Repository. (n.d.). Available: <http://sir.unl.edu/portal/index.php>

Solanki, K., Singh, Y., Dalal, S., & Srivastava, P. (2016). Test Case Prioritization: An Approach Based on Modified Ant Colony Optimization. *Emerging Research in computing, information, communication and applications*, 3, 213–223.

Srikanth, H., Cashman, M., & Cohen, M. B. (2016). Test case prioritization of build acceptance tests for an enterprise cloud application: An industrial case study. *Journal of Systems and Software*, 119, 122–135. doi:10.1016/j.jss.2016.06.017

Srikanth, H., Hettiarachchi, C., & Do, H. (2016). Requirements Based Test Prioritization Using Risk Factors. *Information and Software Technology*, 69(C), 71–83. doi:10.1016/j.infsof.2015.09.002

- Strandberg, P. E., & Sundmark, D. (2016). Experience Report: Automated System Level Regression Test Prioritization Using Multiple Factors. *IEEE 27th International Symposium on Software Reliability Engineering*, 12–23. doi:10.1109/ISSRE.2016.23
- Sujata & Purohit. (2017). *Classification model for test case prioritization techniques*. 2017 International Conference on Computing, Communication and Automation, Greater Noida, India.
- Tahat, L., & Korel, B. (2017). *State-based models in regression test suite prioritization* (Vol. 25). Software Qual J.
- Tahvili, S., Afzal, W., Saadatmand, M., & Bohlin, M. (2016). *Towards earlier fault detection by value-driven prioritization of test cases using fuzzy TOPSIS* (Vol. 448). Information Technology. doi:10.1007/978-3-319-32467-8_65
- Thomas, S. W., Hemmati, H., Hassan, A. E., & Blostein, D. (2012). Static test case prioritization using topic models. *Empirical Software Engineering*, 19(1), 182–212. doi:10.1007/s10664-012-9219-7
- Ufuktepe, E., & Tuglular, T. (2016). Automation Architecture for Bayesian Network Based Test Case Prioritization and Execution. *IEEE 40th Annual Computer Software and Applications Conference (COMPSAC)*, 52–57. doi:10.1109/COMPSAC.2016.71
- Wang, A., & Ali, A. (2016). Enhancing Test Case Prioritization in an Industrial Setting with Resource Awareness and Multi-Objective Search. *Proceedings of the 38th International Conference on Software Engineering Companion*, 182–191. doi:10.1145/2889160.2889240
- Wang, S., Nam, J., & Tan, L. (2017). QTEP: Quality-Aware Test Case Prioritization. *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, 523–534. doi:10.1145/3106237.3106258
- Wang, Y., Zhao, X., & Ding, X. (2015). An effective test case prioritization method based on fault severity. *6th IEEE International Conference on Software Engineering and Service Science (ICSESS)*, 737–741. doi:10.1109/ICSESS.2015.7339162
- Xia, X., Gong, L., Le, T.-D. B., Lo, D., Jiang, L., & Zhang, H. (2016, March). Diversity maximization speedup for localizing faults in single-fault and multi-fault programs. *Automated Software Engineering*, 23(1), 43–75. doi:10.1007/s10515-014-0165-z
- Yadav, D. K., & Dutta, S. (2016). Test case prioritization technique based on early fault detection using fuzzy logic. *3rd International Conference on Computing for Sustainable Global Development [INDIACom]*, 1033–1036.
- Yoo, S., & Harman, M. (2007). Regression Testing Minimisation, Selection and Prioritisation: A Survey. *Test Verif Reliab*, 0, 1–7.
- Yoo, S., & Harman, M. (2012). Regression testing minimization, selection and prioritization: A survey. *Software Testing, Verification & Reliability*, 22(2), 67–120. doi:10.1002/stv.430
- Yoon, H., & Choi, B. (2011). A Test Case Prioritization Based on Degree of Risk Exposure and Its Empirical Study. *International Journal of Software Engineering and Knowledge Engineering*, 21(2), 191–209. doi:10.1142/S0218194011005220
- Yoon, M., Lee, E., Song, M., & Choi, B. (2012). A Test Case Prioritization through Correlation of Requirement and Risk. *Journal of Software Engineering and Applications*, 5(10), 823–836. doi:10.4236/jsea.2012.510095
- Yu, Y. T., & Lau, M. F. (2012). Fault-based test suite prioritization for specification-based testing. *Information and Software Technology*, 54(2), 179–202. doi:10.1016/j.infsof.2011.09.005
- Yuan, F., Bian, Y., Li, Z., & Zhao, R. (2015). Epistatic Genetic Algorithm for Test Case Prioritization. *International Symposium on Search Based*. doi:10.1007/978-3-319-22183-0_8
- Zhang, L., Hao, D., Zhang, L., Rothmel, G., & Mei, H. (2013). Bridging the gap between the total and additional test-case prioritization strategies. *Proceedings - International Conference on Software Engineering*, 192–201. doi:10.1109/ICSE.2013.6606565
- Zhao, X., Wang, Z., Fan, X., & Wang, Z. (2015). A Clustering-Bayesian Network Based Approach for Test Case Prioritization. *IEEE 40th Annual Computer Software and Applications Conference (COMPSAC)*, 542–547. doi:10.1109/COMPSAC.2015.154

APPENDIX

This appendix section presents Table 12 - Table 14.

Table 12. Result Quality Scores of Selected Studies

References	Q1(0.5)	Q2(0.5)	Q3(0.5)	Q4(0.5)	Q5(1)	Q6(1)	Score
Liang et al. (2018)	0.5	0.5	0	0	1	1	3
Elbaum et al. (2014)	0.5	0.5	0.5	0.5	1	1	4
S. Mostafa et al. (2017)	0.5	0.5	0	0	1	1	3
Yoo, S., & Harman (2007)	0.5	0.5	0.5	0.5	1	1	4
Singh et al. (2012)	0.5	0.5	0	0	1	1	3
Thomas et al. (2012)	0.5	0.5	0.5	0.5	1	1	4
Aman et. al. (2018)	0.5	0.5	0	0	1	1	3
Strandberg et. al. (2016)	0.5	0.5	0.5	0.5	0.5	1	3.5
Sampath et al. (2013)	0.5	0.5	0.5	0.5	1	1	4
M. Khanna et al. (2018)	0.5	0.5	0.5	0.5	1	1	4
A. B. Sanchez et al. (2014)	0.5	0.5	0.5	0.5	1	1	4
M. Khanna et al. (2019)	0.5	0.5	0.5	0.5	1	1	4
Mei et al. (2015)	0.5	0.5	0.5	0.5	1	1	4
D. Flemstrom et al. (2018)	0.5	0.5	0.5	0.5	1	1	4
Fang et al. (2014)	0.5	0.5	0.5	0.5	1	1	4
C. Fang et al. (2017)	0.5	0.5	0.5	0.5	1	0.5	3.5
Miranda & Bertolino (2017)	0.5	0.5	0.5	0.5	1	1	4
Tahat et al. (2017)	0.5	0.5	0.5	0.5	1	1	4
Korel et al. (2007)	0.5	0.5	0.5	0.5	1	1	4
Mohd-Shafie ML et al. (2020)	0.5	0.5	0.5	0.5	1	0.5	3.5
Maheswari et al. (2015)	0.5	0.5	0.5	0.5	1	1	4
M. Azizi and H. Do (2018)	0.5	0.5	0.5	0.5	1	1	4
Lou et al. (2016)	0.5	0.5	0.5	0.5	1	1	4
D. Di Nucci et al. (2020)	0.5	0.5	0.5	0.5	1	1	4
Yuan et al. (2015)	0.5	0.5	0.5	0.5	0.5	1	3.5
C. Catal, et al. (2012)	0.5	0.5	0.5	0.5	1	1	4
T. Noguchi et al. (2019)	0.5	0.5	0.5	0.5	1	1	4
A. Kaur & S. Goyal (2011)	0.5	0.5	0.5	0.5	1	1	4
Ahmad et al. (2018)	0.5	0.5	0.5	0.5	1	1	4
Jun et al. (2011)	0.5	0.5	0.5	0.5	1	1	4
Sabharwal et al. (2010)	0.5	0.5	0.5	0.5	1	1	4
Li et al. (2007)	0.5	0.5	0.5	0.5	1	1	4

continued on following page

Table 12. Continued

References	Q1(0.5)	Q2(0.5)	Q3(0.5)	Q4(0.5)	Q5(1)	Q6(1)	Score
Wang et al. (2016)	0.5	0.5	0.5	0.5	1	1	4
Solanki et al. (2016)	0.5	0.5	0.5	0.5	0.5	1	3.5
Anuradha et al. (2018)	0.5	0.5	0.5	0.5	1	1	4
Gao et al. (2015)	0.5	0.5	0.5	0.5	1	0.5	3.5
Ledru et al. (2012)	0.5	0.5	0.5	0.5	1	1	4
Y. Bian et al. (2017)	0.5	0.5	0.5	0.5	1	1	4
S. Eghbali and L. Tahvildari (2016)	0.5	0.5	0.5	0.5	1	1	4
Jones et al. (2003b)	0.5	0.5	0.5	0.5	1	1	4
Di Nardo et al. (2013)	0.5	0.5	0.5	0.5	1	1	4
Nardo et al. (2015)	0.5	0.5	0.5	0.5	1	0.5	3.5
Mostafa et al. (2020)	0.5	0.5	0.5	0.5	1	1	4
Zhang et al. (2014)	0.5	0.5	0.5	0.5	1	1	4
Hao et al. (2013)	0.5	0.5	0.5	0.5	0.5	1	3.5
Hao et al. (2016)	0.5	0.5	0.5	0.5	1	1	4
Fang et al. (2012)	0.5	0.5	0.5	0.5	1	1	4
Jones et al. (2003a)	0.5	0.5	0.5	0.5	1	1	4
Bryce et al. (2011)	0.5	0.5	0.5	0.5	1	1	4
Krishnamoorthi et al. (2009)	0.5	0.5	0.5	0.5	1	1	4
Anderson et al. (2019)	0.5	0.5	0.5	0.5	1	1	4
Tahvili et al. (2016)	0.5	0.5	0.5	0.5	1	1	4
Alves et al. (2016)	0.5	0.5	0.5	0.5	1	1	4
Alves et al. ()	0.5	0.5	0.5	0.5	1	1	4
Mei et al. (2015)	0.5	0.5	0.5	0.5	1	1	4
Mei et al. (2015)	0.5	0.5	0.5	0.5	1	1	4
Jiang, B., et al. (2012)	0.5	0.5	0.5	0.5	1	1	4
Do et al., 2010 (2010)	0.5	0.5	0.5	0.5	1	1	4
Yu, Y. T., & Lau, M. F. (2012)	0.5	0.5	0.5	0.5	1	1	4
Ma et al. (2016)	0.5	0.5	0.5	0.5	1	1	4
Muthusamy & Seetharaman (2014)	0.5	0.5	0.5	0.5	1	1	4
Arafeen et al. (2013)	0.5	0.5	0.5	0.5	1	1	4
M. Yoon et al. (2018)	0.5	0.5	0.5	0.5	1	1	4
Yoon et al. (2012)	0.5	0.5	0.5	0.5	1	1	4
Hettiarachchi et al. (2014)	0.5	0.5	0.5	0.5	1	1	4
Cho et al. (2016)	0.5	0.5	0.5	0.5	1	1	4
Srikanth et al. (2016)	0.5	0.5	0.5	0.5	1	1	4
Marijan et al. (2013)	0.5	0.5	0.5	0.5	1	1	4

continued on following page

Table 12. Continued

References	Q1(0.5)	Q2(0.5)	Q3(0.5)	Q4(0.5)	Q5(1)	Q6(1)	Score
Lin et al. (2013)	0.5	0.5	0.5	0.5	1	1	4
Khalilian et al. (2012)	0.5	0.5	0.5	0.5	1	1	4
Huang et al. (2012)	0.5	0.5	0.5	0.5	1	1	4
Hettiarachchi et al. (2016)	0.5	0.5	0.5	0.5	1	1	4
Alves et al. (2016)	0.5	0.5	0.5	0.5	1	1	4
Yoon, H., & Choi, B. (2011)	0.5	0.5	0.5	0.5	1	1	4
Felderer et al. (2014)	0.5	0.5	0	0	1	1	3
Zhao et al. (2015)	0.5	0.5	0.5	0.5	1	1	4
Mirarab & Tahvildari (2007)	0.5	0.5	0.5	0.5	1	1	4
Ufuktepe et al. (2016)	0.5	0.5	0.5	0.5	1	1	4
Mirarab & Tahvildari (2008)	0.5	0.5	0.5	0.5	1	1	4
Elbaum et al. (2004)	0.5	0.5	0.5	0.5	1	1	4
Srikanth et al.(2016)	0.5	0.5	0.5	0.5	1	1	4
Arafeen et al. (2013)	0.5	0.5	0.5	0.5	1	1	4
Jiang et al. (2015)	0.5	0.5	0.5	0.5	1	1	4
G. Kapfhammer and M. Soffa (2007)	0.5	0.5	0.5	0.5	1	1	4
D. K. Yadav and S. Dutta (2016)	0.5	0.5	0.5	0.5	1	1	4
Laali et al. (2016)	0.5	0.5	0.5	0.5	1	1	4
Xia et al. (2016)	0.5	0.5	0.5	0.5	1	1	4
Fu et al. (2016)	0.5	0.5	0.5	0.5	1	1	4
Khanna et al. ()	0.5	0.5	0.5	0.5	1	1	4

Table 13. The Advantages and Limitation of TCP Approaches

Approach	Domination	Robustness
GA (Search-based)	#Useful for coverage and fault detection (2012).	#Due to mutation, the execution speed is slow.
Greedy (Search-based)	#In comparison to other search algorithms, it provides high coverage and good performance (Li et al., 2007)	#Different variants of search-based algorithm are employed for regression testing (Li et al., 2007).
Ant Colony Optimization (Search-based)	#ACO method is beneficial for TCP approach due to usage of multiple factors in combination (Gao et al., 2015).	#Total coverage might be an issue for ACO (Gao et al., 2015) as ants tend to follow the shorter path.
String Distance (Search-based)	#It detects and kill the strongest mutants in regression testing (Ledru et al., 2012)	#Optimum result cannot be produced, when there two optimal sequences are produced (Ledru et al., 2012)

continued on following page

Table 13. Continued

Approach	Domination	Robustness
Coverage Based	#Full coverage is performed wherein all possible fault (Mostafa et al., 2017) might be detected (Nardo et al., 2015) - (Hao et al., 2014), (). #In terms of APFD (Hao et al., 2014), coverage approaches provide good prediction accuracy.	#Fault detection accuracy is prone to human error (Tahvili et al., 2016). #Faults might not be well defined (Yu & Lau, 2012)
Fault-based	#Has high possibility to detect the faults in a system (Tahvili et al., 2016), (Yu & Lau, 2012) (including hidden faults) #Guaranteed to be free from stated faults (Wang et al., 2015), (Jiang et al., 2012).	#Accuracy of fault detection is subjective and is prone to human error (Tahvili et al., 2016). #Prescribed faults might not be well defined (Yu & Lau, 2012).
Requirement-based	#Use the following information viz., customer priority, error prone value, volatility value, and execution difficulties as prioritization criteria (Srikanth, Hettiarachchi, & Do, 2016). #Claim to gain high efficiency (Jones & Harrold, 2003b), (Wang et al., 2017).	#Due to the human involvement, there is higher possibility of mistakes (Srikanth, Hettiarachchi, & Do, 2016). # Static data may be unreliable due to requirement of perfect mapping (Muthusamy & K, 2014).
Risk-based	#Has high possibility to determine specific error regarding risk assessment in a system (Hao et al., 2014), (Anderson et al., 2019) #Effective in detection of severe faults at an early stage (Hao et al., 2014), (Krishnamoorthi & Sahaaya Arul Mary, 2009)	# Its requires the involvement of human expert (Anderson et al., 2019)-(Tahvili et al., 2016).
History-based	#Utilize historical data utilization like execution's count to reveal fault (Jones & Harrold, 2003a). #Accelerate rate of fault-detection (Jones & Harrold, 2003a) and (Bryce et al., 2011)	#This approach has several bottlenecks such as availability of historic information and small number of defective information.
Bayesian-Network-based	#This uses different kinds of information such as fault proneness and coverage info etc. to predict the probability of fault detection using Bayesian network model (), (Jiang et al., 2012)	# Needs to improve the efficacy of the algorithm that is time consuming (Jiang et al., 2012).
Cost-Effective-based	#This promotes cost awareness leading to cost reduction (Do et al., 2010).	#Offers several bottlenecks like availability of historic information and small number of irrelevant information

Table 14. Major Process proposed by different Researchers

No.	Source	Process	
1	Fang, C., Chen, Z., & Xu, B. (2012)	1)	Described several logic coverage criteria.
		2)	Evaluated Fault detection rate.
		3)	evaluated an efficacy of the proposed approach based on CI.
		4)	Calculated Test Case Priorities
2	Bryce et al. (2011)	1)	Most unique area including test cases were selected for study.
		2)	Randomly selected test cases Iterates for tie breaking
		3)	Test with the newest pairs iterates over till completion.
3	Eghbali, S, & Tahvildari, L. (2016)	1)	Selected the highest coverage test cases.
		2)	Select next highest coverage test case that has not been covered yet.
		3)	Repeat step 2
		4)	Random selection for ties and repeat till completion as per the order.
4	Tahvili et al (2016)	1)	Determined the criteria.
		2)	Found the parameter score as per the criteria.
		3)	TCP is done using the second step.
5	Wang et al. (2015)	1)	Fault severity is computed.
		2)	Fault severity with highest score for test cases is used to record the covered test cases and faults.
		3)	Fault severity of the remaining test cases is updated.
		4)	Repeat step 2 to 4 until all the test cases are sorted
6	Jiang, B, et al. (2012)	1)	Cost of discovering fault localization is calculated.
		2)	Cost to find second fault localization is computed.
		3)	Repeat for third and so on.
		4)	Find Mean of the cost for localizing the faults.
7	Yu, Y T, & Lau, M.F. (2012)	1)	Find fault from prescribe model.
		2)	Using MUMCUT, test cases are generated
		3)	Evaluated the efficacy of the developed model.
8	Do et al. (2010)	1)	Find coverage information for all object programs.
		2)	Test suites are reordered.
		3)	Fault detection rate is computed
		4)	cost prioritization is done based on information gathered regarding fault detection
9	Srikanth et al. (2016)	1)	Analyzed the failure of historical fields.
		2)	Identified rare use cases.
		3)	Use cases are tagged.
		4)	Prioritized the use cases.
10	Khalilian et al. (2012)	1)	First analyzed the Historical data.
		2)	Collected Test case coverage data as per the coverage criteria.
		3)	Percentage code coverage is then used for prioritization.
		4)	Final data is recorded.

continued on following page

Table 14. Continued

No.	Source	Process	
11	Ma, T, Zeng, H, & Wang, X. (2016)	1)	Computed the priorities of test cases.
		2)	Selected test cases with higher scores.
		3)	Prioritized and recalculated the priorities.
		4)	Repeat the steps to cover all the requirements.
12	Srikanth et al. (2016)	1)	Computed customer priority scores as well as fault scores.
		2)	Computed the average of all scores
		3)	Average value of scores is divided by summation of both scores calculated in step 1 to find weights.
		4)	Compute total scores by multiplying each one with a weight factor.
13	Arafeen et al. (2013)	1)	Did the requirement clustering
		2)	Did the requirement mapping.
		3)	Done the prioritization of clusters.
		4)	Whole prioritization done
14	Krishnamoorthi et al. (2009)	1)	Identification of factors done.
		2)	Computed total requirement and test cases.
		3)	Calculated the factor values.
		4)	Computed requirement weights.
		5)	Test case analyzed and mapped to the requirements.
		6)	Recalculated the requirement weights.
		7)	Sorted the Test cases.
15	Hettiarachchi et al. (2014)	1)	Determined the requirement criteria
		2)	Found the Requirement priorities.
		3)	Did the Mapping of Defect and failure to requirement
		4)	Analyzed the requirement complexity.
		5)	Run the test cases.
16	Hettiarachchi et al. (2016)	1)	Estimated the risk.
		2)	Calculated the Risk requirement.
		3)	Calculate the Risk items.
		4)	Start off the TCP process
17	Hettiarachchi et al. (2014)	1)	Did the risk estimation.
		2)	Performed the risk weight calculation.
		3)	Calculated the risk value.
		4)	Evaluation of extra factors
		5)	Start off the TCP process
18	Yoon et al. (2012)	1)	Identified the risk items.
		2)	Estimated the Risk values
		3)	Estimated the Test case priority
		4)	Test case priority is used for prioritization

Harendra Singh is a PhD Scholar of Computer Science & Engineering at Dr APJ Abdul Kalam Technical University, Lucknow, U.P., India and has obtained M.Tech. degree in Computer Science & Engineering in 2016 from Dr APJ Abdul Kalam Technical University, Lucknow, U.P. His research interest includes Test case selection and prioritization in regression testing using nature inspired algorithms. He has published various research papers in Journals and presented in conferences. Presently working as Assistant Professor at Greater Noida Institute of Engineering & Technology in the Department of CSE, Greater Noida, and having around 17 years of teaching experience.

Laxman Singh obtained his B. Tech in Electronics and Communication Engineering from C.R. State (Govt.) College of Engineering, Murthal, Sonapat (Haryana) and M.Tech in Instrumentation and Control from M.D. University, Haryana, India in 2004 and 2009 respectively. He received his PhD degree from Jamia Millia Islamia (a central Govt. of India University) in 2016. Presently he is working as Associate Professor in the Department of Electronics & Communication Engineering at Noida Institute of Engineering & Technology (NIET), Greater Noida. He has total teaching experience of more than seventeen years. Dr. Laxman Singh has published about 70 research articles in the field of image processing, AI, machine learning, and robotic path planning in various refereed international/national journals as well as in international conferences of repute. He has authored two books; one on "Practical machine learning with Python", and another on "Electronic Circuits and applications", published by GBS Publisher, New Delhi. His current research interests are in the areas of Wavelet analysis, Artificial Intelligence, Image processing, Optimization techniques, and robotic path planning.

Shailesh Tiwari is currently working as Additions Director in Krishan Institute of Engineering and Technology, Ghaziabad, India. He is an Alumnus of Motilal Nehru National Institute of Technology Allahabad, India. His primary areas of research are software testing, implementation of optimization algorithms and machine learning techniques in various engineering problems. He has published more than 80 publications in international journals and in proceedings of international conferences of repute. He has edited special issues of several Scopus, SCI and E-SCI-indexed journals. He has also edited several books published by Springer. He has published 4 Indian patents as IPRs. He has organized several international conferences under the banner of IEEE, ACM and Springer. He is a Senior Member of IEEE, Member of IEEE Computer Society, Fellow of Institution of Engineers (FIE).